

MATERIAL AUTORAL

Processamento de Dados e Fine-Tuning de Modelos

Disciplina 9 • Engenharia de Software com IA Aplicada

Autoria: Ahirton Lopes

Como usar este material

Este material foi reorganizado a partir do conteúdo integral das aulas fornecidas. O texto-base permanece fiel à transcrição; a edição acrescenta estrutura didática, navegação interna, diagramas conceituais e atividades de revisão sem substituir as afirmações apresentadas pelo professor.

PRINCÍPIO DE FIDELIDADE

- O conteúdo conceitual das aulas é preservado; a edição atua em organização, hierarquia visual e correções editoriais mínimas.
- Diagramas reorganizam relações explicitamente apresentadas nas aulas e não são apresentados como medições quantitativas.
- Gráficos numéricos não são criados quando a transcrição não fornece valores adequados para comparação.
- Números, casos e exemplos quantitativos permanecem vinculados ao contexto em que aparecem nas aulas.

COMO ESTUDAR CADA AULA

- Leia os objetivos e observe o fluxo visual da unidade antes de avançar.
- Estude o conteúdo integral da aula, agora organizado para facilitar a leitura.
- Use o bloco Na prática para transformar conceitos em uma pequena aplicação ou registro verificável.
- Responda ao checkpoint sem consultar o texto; retorne aos trechos em que houver dúvida.
- Conclua pela revisão da unidade e utilize os links internos para voltar diretamente às aulas que precisarem de reforço.

Introdução da disciplina

Nesta disciplina, eu começo por uma pergunta que precisa vir antes de qualquer dataset, API de treinamento ou escolha de hiperparâmetro: será que fine-tuning é realmente a ferramenta certa para o problema que eu tenho em mãos?

A disciplina anterior deixou uma situação em aberto de propósito. No Trial Forge, a síntese de CSR sempre era encaminhada para um modelo mais caro por uma regra fixa, porque errar naquela etapa custava demais. Só que uma regra fixa não significa que o problema esteja resolvido. A pergunta que abre esta nova disciplina é justamente se poderíamos ter um modelo menor, especializado naquela tarefa, barato e preciso o suficiente para deixar de escalar todas as chamadas.

Essa é a porta de entrada para Processamento de Dados e Fine-Tuning de Modelos. Antes de ensinar a preparar dados ou rodar um treinamento, eu quero organizar a decisão. Fine-tuning é uma das ferramentas mais caras e mais lentas de iterar entre as opções disponíveis. Por isso, deveria ser uma das últimas alternativas consideradas, e não a primeira.

Percurso da disciplina

- Unidade 1: Quando Fazer Fine-Tuning (Decision Framework).
- Unidade 2: Preparação de Datasets para Fine-Tuning.
- Unidade 3: Fine-Tuning via API (upload e train).

Mapa da disciplina

Navegue por uma unidade ou aula. Os títulos abaixo são links internos para os respectivos pontos do material.

Unidade 1: Quando Fazer Fine-Tuning (Decision Framework) [↗ ir para a unidade](#)

[Aula 1: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-1\)](#)

[Aula 2: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-2\)](#)

[Aula 3: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-3\)](#)

Unidade 2: Preparação de Datasets para Fine-Tuning [↗ ir para a unidade](#)

[Aula 1: Preparação de Datasets para Fine-Tuning \(PT-1\)](#)

[Aula 2: Preparação de Datasets para Fine-Tuning \(PT-2\)](#)

Unidade 3: Fine-Tuning via API (upload e train) [↗ ir para a unidade](#)

[Aula 1: Fine-Tuning via API \(upload e train\) \(PT-1\)](#)

[Aula 2: Fine-Tuning via API \(upload e train\) \(PT-2\)](#)

[Aula 3: Fine-Tuning via API \(upload e train\) \(PT-3\)](#)

[Aula 4: Fine-Tuning via API \(upload e train\) \(PT-4\)](#)

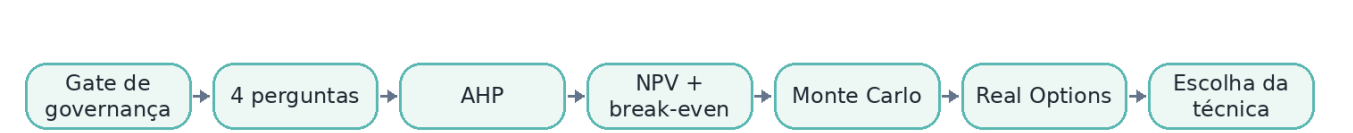
[Aula 5: Fine-Tuning via API \(upload e train\) \(PT-5\)](#)

Unidade 1

Quando Fazer Fine-Tuning (Decision Framework)

Decisão estruturada sobre quando fine-tuning faz sentido, combinando governança, quatro condições necessárias, análise multicritério e avaliação econômica antes de escolher uma técnica de treinamento.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Quando Fazer Fine-Tuning (Decision Framework) (PT-1)
2	Quando Fazer Fine-Tuning (Decision Framework) (PT-2)
3	Quando Fazer Fine-Tuning (Decision Framework) (PT-3)

Aula 1 — Quando Fazer Fine-Tuning (Decision Framework) (PT-1)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-1)”.
- Relacionar Fine-tuning e RAG às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Fine-tuning	• RAG
• Decision Framework	• Context engineering
• Gate de governança	• Tarefa estreita e repetida

Conteúdo da aula

O caso da Amplitude Seguros

Para aplicar esse raciocínio, eu utilizo o caso da Amplitude Seguros. A empresa nasceu como cooperativa de produtores rurais, dividindo risco e, décadas depois, se transformou em sociedade anônima e ampliou a atuação para linhas como automóvel, saúde e empresarial.

Essa origem ainda influencia a cultura da companhia. Existe uma tendência forte à decisão colegiada e à aversão a risco. Nesse ambiente, Camila Andrade, head de dados e IA, já não precisa convencer o comitê de que IA generativa importa. Prompting e RAG já aparecem em produção em diferentes frentes.

A questão agora é mais difícil: quando vale a pena ir além de prompt e recuperação de contexto e assumir o custo de treinar um modelo próprio? Em uma organização acostumada a evitar gastos desnecessários, essa decisão precisa ser fundamentada, não movida apenas pela vontade de adotar uma técnica mais sofisticada.

O mito mais importante sobre fine-tuning

Antes do framework, eu preciso derrubar o erro mais comum de quem chega ao fine-tuning pela primeira vez: a ideia de que ele serve para ensinar fatos novos ao modelo.

Esse não é o papel principal do fine-tuning. Fine-tuning ensina comportamento, formato, padrão de resposta, tom e estrutura. Ele ajuda o modelo a responder de uma maneira consistente para uma classe específica de solicitações.

Quem resolve conhecimento novo é o RAG. Nesse caso, o conhecimento fica em uma fonte externa que o modelo consulta a cada chamada. O texto relevante é recuperado e inserido no contexto. O modelo em si não muda.

Confundir esses dois problemas é caro porque pode levar uma equipe a treinar um modelo inteiro quando o que realmente precisava era de uma fonte atualizável de conhecimento.

RAG e fine-tuning resolvem problemas diferentes

A diferença mecânica explica por que essa separação precisa ser tão rígida.

No RAG, o modelo permanece o mesmo. Na hora da requisição, o sistema procura a informação certa em um índice e coloca esse conteúdo no prompt. Se eu alterar o índice, a próxima chamada já pode utilizar o conhecimento novo.

No fine-tuning, eu pego um modelo previamente treinado e continuo o treinamento usando meus próprios exemplos. Durante esse processo, os pesos internos são ajustados por gradiente descendente. Mais adiante na disciplina, nós vamos observar esse treinamento acontecendo de verdade, inclusive acompanhando a perda cair ao longo das iterações.

Isso torna o RAG muito mais reversível. Eu posso trocar ou corrigir a base de conhecimento e ver o efeito imediatamente. No fine-tuning, o comportamento aprendido passa a fazer parte do próprio modelo treinado. Ele não depende de um texto externo ser reinserido em toda chamada para aparecer.

Casos que mostram a diferença na prática

A aula apresenta exemplos de mercado para mostrar os dois lados.

Em um caso ligado ao ecossistema do Ray, textos de Shakespeare foram modificados substituindo sistematicamente o nome Romeo por Bob. Depois do fine-tuning, quando o modelo recebeu uma pergunta sobre o amado de Julieta e ainda recebeu a pista de que o nome começava com R, continuou respondendo Romeo. Mesmo uma substituição tão direta não se transformou de forma confiável em conhecimento factual novo.

Outro estudo citado, publicado por pesquisadores da Microsoft em 2024, comparou fine-tuning e RAG para injeção de conhecimento novo em diferentes modelos. No Llama 2, o fine-tuning isolado chegou a piorar a acurácia em fatos recentes, enquanto o RAG elevou a acurácia de aproximadamente 35% para quase 60%.

O ponto não é concluir que fine-tuning nunca serve. O ponto é perceber que ele estava sendo aplicado ao problema errado.

No outro lado está a Indeed. A plataforma já havia validado o comportamento de explicações de recomendação de vagas com técnicas como few-shot prompting. Depois, usou fine-tuning em um modelo menor para manter a qualidade com menor custo de inferência. O caso citado registra uma redução de cerca de 60% no consumo de tokens em uma escala de milhões de mensagens por mês.

A distinção que eu quero manter é simples: RAG resolve conhecimento. Fine-tuning resolve comportamento repetido em volume e pode tornar esse comportamento mais barato do que reconstruir um prompt grande a cada chamada.

A primeira pergunta: a tarefa é estreita e repetida?

A primeira pergunta do framework é se a tarefa possui uma forma estreita e repetida ou se ela é aberta e variável.

Fine-tuning funciona melhor quando eu consigo descrever a tarefa em uma frase que continua válida para praticamente todos os casos. Um exemplo seria extrair segurado, placa e valor de um documento de sinistro. A entrada muda, mas a natureza da tarefa e o contrato de saída continuam reconhecíveis.

Esse é um sinal verde porque existe repetição suficiente para que um comportamento especializado faça sentido.

O sinal vermelho aparece quando a própria natureza da tarefa muda. Em uma chamada eu quero resumir, em outra traduzir, em outra responder uma pergunta aberta e em outra produzir um formato completamente diferente. Nesse cenário, eu não tenho um comportamento estreito para especializar. Um bom prompt tende a ser uma ferramenta melhor.

A segunda pergunta: as alternativas mais baratas já foram esgotadas?

A segunda pergunta é a que muita gente pula: eu já tentei de verdade prompt engineering, RAG, roteamento de modelo e cache de contexto ou de prompt antes de decidir treinar?

A Indeed é importante justamente porque seguiu essa ordem. Primeiro validou que o comportamento funcionava com few-shot prompting. Só depois partiu para fine-tuning com o objetivo de reduzir custos em escala.

Existe um teste prático. Se eu consigo explicar em texto exatamente como quero que o modelo se comporte, primeiro preciso tentar colocar essa descrição em um system prompt bem escrito, utilizando exemplos reais dentro do próprio contexto.

O sinal verde aparece quando esse trabalho já foi feito de maneira séria e ainda assim existe uma falha consistente de qualidade, custo ou latência. O sinal vermelho é considerar fine-tuning antes mesmo de tentar melhorar o prompt adequadamente.

A regra é importante porque treinar não deveria ser uma forma de evitar o trabalho de especificar o comportamento que eu quero.

A terceira pergunta: existem dados suficientes e de qualidade?

A terceira pergunta é se eu tenho exemplos suficientes, diversos e corretos para treinar.

Essa questão será aprofundada no módulo 2, mas precisa entrar no decision framework desde o começo. Sem dados bons, não existe fine-tuning bom.

Suficiente não significa necessariamente milhares de exemplos. Significa possuir exemplos reais e representativos da variação que existe na tarefa-alvo.

Um sinal verde seria já ter um histórico real de entradas e saídas corretas para aquela operação, mesmo que inicialmente sejam apenas algumas dezenas de casos, desde que cubram de forma útil a diversidade do problema.

O sinal vermelho aparece quando eu preciso inventar todo o dataset do zero sem possuir nenhum caso real como referência. Dados artificiais podem aparecer em estratégias mais avançadas de expansão, mas não deveriam ser a única fundação de um dataset quando eu sequer conheço a variação real do processo que quero modelar.

A quarta pergunta: a tarefa é estável?

A quarta pergunta avalia se a tarefa é estável o suficiente para justificar o custo de especialização.

Se o formato de saída muda toda semana, cada mudança importante pode exigir revisão do dataset, nova avaliação e, em alguns casos, novo treinamento. Esse custo de manutenção precisa ser considerado antes do primeiro ciclo de fine-tuning.

O sinal verde é ter um esquema de saída tratado como contrato. Ele muda raramente e, quando muda, isso acontece por uma decisão deliberada de produto ou arquitetura.

O sinal vermelho aparece quando cada área da empresa quer uma resposta diferente e ainda não existe um contrato consolidado. Treinar antes dessa definição é tentar acertar um alvo que ainda está se movendo.

Nos módulos seguintes, eu vou transformar esse custo em algo mais concreto, inclusive mostrando o custo real de um ciclo de treinamento.

A escada de customização ficou maior

Existe um adendo importante porque o mercado evoluiu. Hoje os modelos de fronteira aceitam janelas de contexto muito maiores do que alguns anos atrás. Modelos de raciocínio também conseguem gastar mais tempo de inferência antes de responder, e o custo por token caiu em muitos cenários.

Isso não muda a diferença entre RAG e fine-tuning. O primeiro continua ligado ao conhecimento recuperado. O segundo continua ligado ao comportamento aprendido. O que mudou foi a quantidade de alternativas intermediárias que eu posso tentar antes de treinar.

Uma delas é context engineering. Aqui eu penso em tudo que entra no contexto do modelo, não apenas no prompt escrito manualmente. Histórico de conversa, documentos recuperados, saída de ferramentas e exemplos escolhidos dinamicamente fazem parte dessa disciplina maior. RAG é uma técnica dentro desse conjunto.

Outra alternativa são agent skills, pacotes de instruções e, em alguns casos, código que um agente pode carregar sob demanda quando precisa de uma capacidade específica. Eles não precisam permanecer no contexto o tempo inteiro e também não exigem alteração dos pesos do modelo.

A escada passa, então, por prompt, context engineering, RAG, agent skills e outras técnicas antes de chegar ao fine-tuning.

Onde fine-tuning continua ganhando

Mesmo com modelos excelentes e contexto grande, existem situações em que fine-tuning continua tendo vantagens claras.

A primeira é volume. Se eu preciso reconstruir um contexto grande em milhões de chamadas, o custo de tokens e latência pode superar o investimento de ter parte daquele comportamento embutido em um modelo especializado.

A segunda é consistência de formato. Em tarefas em que o esquema de saída precisa ser extremamente estável, fine-tuning pode reduzir a dependência de repetir instruções de formato em todo prompt.

Ainda assim, isso não elimina a validação determinística. A própria OpenAI oferece mecanismos como Structured Outputs para garantir aderência a um esquema JSON. Fine-tuning pode aumentar a consistência, mas não deveria ser a única camada de proteção.

A terceira situação aparece quando eu preciso rodar um modelo pequeno, local, fora da nuvem e sem acesso a um contexto enorme ou a skills externas. Essa é uma das situações que será demonstrada mais adiante com um modelo pequeno executando localmente.

O checklist das quatro perguntas

O decision framework é transformado em um checklist com as quatro perguntas, os sinais verdes, os sinais vermelhos e uma primeira aplicação ao caso da Amplitude Seguros.

As perguntas são: a tarefa é estreita e repetida; as alternativas mais baratas já foram esgotadas; existem dados suficientes, diversos e de qualidade; e a tarefa é estável o bastante para não criar uma esteira de retreino constante.

Se uma resposta importante cai no vermelho, eu interrompo a recomendação de fine-tuning naquele ponto. Não faz sentido forçar as outras perguntas apenas para justificar uma decisão que já começou errada.

O checklist não substitui julgamento. Ele organiza o julgamento. Nos próximos vídeos, ele deixa de ser somente uma tabela e passa a ser aplicado a casos distintos da Amplitude Seguros, até chegar a uma implementação executável que recomenda caminhos diferentes para problemas diferentes.

O portão de governança

Existe ainda uma quinta preocupação que as quatro perguntas não cobrem completamente: governança.

No caso da Amplitude, um dos cenários envolve recibos médicos e dados de saúde sensíveis. Treinar um modelo com esse tipo de informação não é a mesma decisão que consultar um dado externo por RAG.

Quando a informação entra no processo de treinamento, retenção, auditoria e direito ao esquecimento fica mais difícil de operacionalizar. Por isso, a governança passa a funcionar como um portão anterior às quatro perguntas nos capítulos seguintes.

Se o dado não pode ser utilizado de forma adequada para treinamento, pouco importa que a tarefa seja estreita, estável e tenha volume. O candidato não deveria avançar para fine-tuning.

Fine-tuning também cria novas responsabilidades

Antes de seguir para os estudos de caso, eu deixo registrado que fine-tuning não encerra problemas. Ele troca parte dos problemas de inferência por problemas de dados e manutenção.

Dados ruins não avisam que estão errados. Eles viram comportamento aprendido. Um dataset pequeno demais ou uma configuração agressiva pode levar a overfitting, em que o modelo memoriza exemplos em vez de aprender um padrão mais geral.

Cada modelo fine-tunado também vira um artefato que precisa ser versionado, reavaliado e eventualmente retreinado quando o modelo base muda.

E fine-tuning sem uma avaliação rigorosa contra um baseline é especialmente perigoso porque pode gerar falsa confiança. Uma melhora aparente em alguns exemplos não prova que a solução funciona melhor no problema real.

As quatro perguntas desta aula servem para filtrar candidatos. Elas não substituem preparação de dados, treinamento, avaliação e governança. Nos próximos capítulos, o mesmo framework será aplicado a casos da Amplitude Seguros que parecem semelhantes na origem, mas não recebem a mesma recomendação quando cada pergunta é analisada com cuidado.

NA PRÁTICA

- Escolha uma tarefa do seu contexto e responda às quatro perguntas do Decision Framework.
- Registre, para cada pergunta, o sinal verde ou vermelho e a evidência que sustenta a classificação.
- Verifique se o gate de governança permite que o caso avance antes de discutir treinamento.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-1)”?
- Qual relação existe entre Fine-tuning e RAG no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- Nesta disciplina, eu começo por uma pergunta que precisa vir antes de qualquer dataset, API de treinamento ou escolha de hiperparâmetro: será que fine-tuning é realmente a ferramenta certa para o problema que eu tenho em mãos?
- A disciplina anterior deixou uma situação em aberto de propósito.
- Essa é a porta de entrada para Processamento de Dados e Fine-Tuning de Modelos.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — Quando Fazer Fine-Tuning (Decision Framework) (PT-2)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-2)”.
- Relacionar condições necessárias e Gate de governança às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Condições necessárias	• Gate de governança
• Representatividade de dados	• Amplitude Auto
• Saúde Empresarial	• Atendimento ao Cliente

Conteúdo da aula

No capítulo anterior, eu apresentei o framework de decisão para avaliar quando fine-tuning realmente faz sentido. Agora eu aplico esse framework a casos concretos da Amplitude Seguros. A ideia é sair da regra decorada e observar como duas tarefas aparentemente parecidas podem receber recomendações diferentes quando eu analiso cada condição necessária de forma explícita.

Vou trabalhar com três casos. Os dois primeiros pertencem à mesma empresa e têm uma origem semelhante: documentos externos, formatos inconsistentes e necessidade de extrair informação de forma confiável. Mesmo assim, Amplitude Auto e Amplitude Saúde Empresarial não chegam à mesma recomendação. O terceiro caso, atendimento ao cliente, é importante porque mostra outra nuance: ter muito dado disponível não significa automaticamente que fine-tuning seja adequado.

O objetivo é aplicar as quatro perguntas com justificativa registrada e acrescentar um gate de governança que roda antes de qualquer ponderação. A implementação executável fica para o próximo capítulo. Aqui, o foco é entender exatamente onde a decisão muda.

As quatro perguntas continuam sendo condições necessárias

O framework continua com as mesmas quatro perguntas. Primeiro, a tarefa é estreita e repetida? Segundo, prompt engineering, RAG e as alternativas mais baratas já foram realmente esgotadas? Terceiro, existem dados suficientes, diversos e representativos para treinamento? Quarto, a tarefa é estável o bastante para justificar o investimento e a manutenção de um modelo fine-tunado?

As quatro precisam estar verdes. Isso não representa uma aprovação arbitrária por unanimidade. Cada pergunta verifica uma condição necessária diferente.

Uma tarefa estreita sem dados suficientes não é um bom candidato. Um grande volume de dados aplicado a uma tarefa aberta também não resolve. Uma tarefa repetitiva que muda de contrato toda semana pode transformar o fine-tuning em uma esteira de retreino.

Por isso, as perguntas não compensam umas às outras. Um sinal vermelho já muda a recomendação, independentemente de quão fortes sejam os sinais verdes restantes.

Relembrando o papel do RAG

Como a segunda pergunta compara fine-tuning com alternativas anteriores, vale relembrar rapidamente o papel do RAG.

No RAG, a pergunta é representada de forma que o sistema consiga localizar trechos semanticamente relevantes em uma base de conhecimento. Esses trechos são recuperados e enviados junto com a solicitação para o modelo. Nada é retreinado.

É justamente por isso que, se prompt e RAG já resolvem o problema com qualidade, custo e latência aceitáveis, fine-tuning não deveria entrar em cena. Eu só avanço para treinamento quando existe uma necessidade que essas técnicas não resolveram de forma suficiente.

O gate de governança vem antes da ponderação

Antes das quatro perguntas, eu adiciono um portão de governança.

Esse gate verifica uma questão binária: os dados podem ser processados por um provedor de fine-tuning do ponto de vista legal e de compliance? Preciso ter uma base legal definida e, quando houver dados de categoria sensível, preciso considerar os acordos exigidos com o provedor, como um DPA adequado ao contexto.

Esse gate não funciona como uma quinta pergunta ponderável. Ele é um blocker. Nenhum volume de dados, ganho financeiro ou pontuação favorável em um método de decisão compensa uma base legal ausente.

Por isso ele precisa rodar antes do AHP e antes da análise econômica. Não faz sentido gastar tempo avaliando retorno financeiro de um caso que juridicamente não pode prosseguir.

Na Amplitude Seguros, Saúde Empresarial carrega uma exigência adicional porque envolve dados de saúde, categoria sensível. Auto e atendimento ao cliente não possuem essa mesma característica no exemplo. Os três casos utilizados na aula passam pelo gate, mas o resultado é verificado caso a caso e fica acompanhado de justificativa.

Governança não termina no gate

Passar no gate não significa que toda a discussão de privacidade esteja resolvida.

Existe um passo técnico adicional entre a autorização de uso e a construção do dataset: higienizar informações pessoalmente identificáveis antes que elas cheguem ao treinamento.

CPF, nome e outros identificadores podem ser memorizados pelo modelo e eventualmente aparecer em uma resposta futura se forem mantidos de forma inadequada no dataset. Esse risco será tratado com mais profundidade no módulo de preparação de dados.

A disciplina antecipa inclusive uma ferramenta específica para higienização, com validação real de CPF pelo dígito verificador, além de materiais complementares sobre privacidade diferencial, fine-tuning federado e risco de memorização.

A separação é importante. O gate responde se o piloto pode prosseguir. Mascaramento, auditoria e demais controles determinam como o dado precisa ser preparado e governado durante o restante do pipeline.

Caso 1: Amplitude Auto

O primeiro caso é Amplitude Auto. A tarefa é extrair segurado, placa e valor de orçamentos de oficina.

O problema é realista porque o formulário principal do sinistro pode até chegar estruturado pela plataforma, mas o documento de apoio vem da oficina. Cada oficina pode utilizar um formato diferente, criado por software próprio, impresso ou até preenchido de maneira menos padronizada.

Antes de analisar o caso, eu deixo uma ressalva: os números financeiros utilizados ao longo do exemplo são ilustrativos. Eles existem para construir uma história econômica coerente e deverão ser substituídos pelos números reais de cada organização quando o framework for utilizado na prática.

Amplitude Auto: pergunta 1

A tarefa é estreita e repetida? Sim.

A saída é sempre a mesma: segurado, placa e valor. Os documentos de entrada variam bastante entre oficinas, mas a variação está na forma de entrada, não na natureza da tarefa.

Esse é exatamente o tipo de situação em que fine-tuning pode aprender um comportamento consistente. A primeira pergunta recebe sinal verde.

Amplitude Auto: pergunta 2

Prompt e RAG já foram esgotados? Sim.

A equipe já trabalhou com prompt bem escrito e RAG em produção. Ainda assim, a variação de layout entre oficinas mantém inconsistências de saída em escala.

O ponto não é que prompt tenha sido ignorado. Pelo contrário, ele já foi testado e validado até o limite em que consegue ajudar. Fine-tuning aparece como próximo degrau porque existe um problema residual de consistência que continua relevante.

A segunda pergunta também fica verde.

Amplitude Auto: perguntas 3 e 4

Existe dado suficiente? Sim. A linha de Auto é madura e possui milhares de sinistros processados historicamente, cobrindo a diversidade real dos orçamentos.

A tarefa também é estável. O sistema espera os mesmos campos e o contrato de saída não muda semanalmente.

Portanto, as perguntas 3 e 4 ficam verdes. O resultado é quatro sinais verdes. Para Amplitude Auto, o framework considera que fine-tuning vale a pena como candidato.

Caso 2: Amplitude Saúde Empresarial

O segundo caso parece muito parecido na superfície. Em Saúde Empresarial, a tarefa é extrair do beneficiário, procedimento e valor de recibos médicos e ainda cruzar essas informações com o cadastro da empresa cliente para identificar se o pedido pertence ao titular ou a um dependente.

A estrutura é semelhante à de Auto, mas o domínio adiciona uma camada regulatória importante porque os documentos carregam dados de saúde.

O gate de governança precisa, portanto, ser tratado com atenção adicional antes de qualquer análise das quatro perguntas.

Saúde Empresarial: perguntas 1 e 2

A primeira pergunta recebe sinal verde. A tarefa é estreita e repetida. O contrato continua previsível: beneficiário, procedimento, valor e resultado do cruzamento entre titular e dependente.

A segunda pergunta também fica verde. A equipe já testou prompt e RAG, seguindo o mesmo tipo de processo de validação utilizado em Auto.

Até aqui, os dois casos parecem equivalentes.

Saúde Empresarial: a pergunta que muda a decisão

A divergência aparece na terceira pergunta: existe dado suficiente, diverso e representativo? Neste momento, não.

Saúde Empresarial é uma linha mais nova da companhia e possui um histórico menor. Existem dados, mas eles ainda não cobrem de forma satisfatória a variedade real de recibos, prestadores e situações.

O problema não é a ausência completa de exemplos. O problema é a representatividade.

A aula coloca números ilustrativos para tornar a diferença concreta. Auto processa aproximadamente 8 mil sinistros por mês, enquanto Saúde Empresarial processa cerca de 1.200. É uma diferença próxima de sete vezes.

O score utilizado no framework acompanha essa distância. Auto recebe aproximadamente 0,9 nesta dimensão, enquanto Saúde Empresarial fica em 0,35, abaixo de um limiar de 0,6 utilizado para separar aprovação e reprovação nesta pergunta.

O framework não cria a diferença. Ele formaliza uma diferença que já existe nos dados disponíveis.

Saúde Empresarial: tarefa estável, mas ainda não

A quarta pergunta volta a ser verde. O contrato de saída é estável e integrado ao mesmo tipo de sistema de sinistros.

O resultado final, portanto, é três perguntas verdes e uma vermelha.

Nesse caso, a recomendação não é que fine-tuning nunca servirá. A recomendação é ainda não.

O gargalo é dado histórico representativo. Se o volume crescer e a variedade do problema passar a estar adequadamente coberta, a decisão poderá ser reavaliada.

Essa é uma diferença importante em relação ao terceiro caso, no qual o problema não pode ser resolvido simplesmente esperando mais dados.

Caso 3: Atendimento ao Cliente

O terceiro caso é atendimento ao cliente.

A tarefa envolve negociar e resolver contestações de sinistro em conversas abertas com segurados, incluindo exceções de cobertura analisadas caso a caso.

Um segurado pode questionar um item de guincho, outro pode discutir uma exceção de cobertura, outro pode precisar de escalonamento para um especialista. O contexto, o histórico, o produto contratado e o tipo de decisão mudam entre atendimentos.

Esse cenário possui mais dados do que os anteriores, mas ainda assim reprova no framework.

Atendimento ao Cliente: a tarefa não é estreita

A primeira pergunta recebe sinal vermelho.

A tarefa é aberta. Não existe uma única estrutura de saída comparável a segurado, placa e valor.

Cada contestação pode exigir uma combinação diferente de interpretação, consulta de cobertura, justificativa, negociação e decisão.

Nesse caso, mais dados não tornam a tarefa estreita. O problema não é a falta de exemplos. É a ausência de um padrão único que faça sentido embutir como comportamento especializado.

Atendimento ao Cliente: alternativas e dados estão fortes

A segunda pergunta fica verde de propósito. Prompt bem escrito, RAG sobre apólices e regras de cobertura e roteamento para atendimento humano em situações sensíveis já estão em produção.

A terceira pergunta também fica verde. Esse é o caso com maior volume de dados entre os três. Existem milhares de conversas por mês, em quantidade superior à de Auto e Saúde Empresarial combinadas.

Essa combinação é importante porque impede uma interpretação simplista do framework. Dado de sobra não basta para justificar fine-tuning.

Atendimento ao Cliente: a tarefa também é instável

A quarta pergunta recebe outro sinal vermelho.

Políticas de cobertura podem mudar com revisões regulatórias e com a criação de novos produtos. Uma exceção aceitável hoje pode deixar de ser válida depois de uma alteração de regra.

Treinar um comportamento fixo sobre uma política que muda com frequência cria risco de obsolescência rápida.

O resultado é diferente de Saúde Empresarial. Ali, o framework diz que ainda não, porque falta representatividade de dados. Em atendimento ao cliente, a recomendação é não usar fine-tuning para essa tarefa como ela está definida, porque os problemas centrais são abertura e instabilidade.

Três casos, três recomendações

Os três casos produzem recomendações diferentes.

Amplitude Auto passa: tarefa estreita, alternativas anteriores esgotadas, dados representativos e contrato estável.

Saúde Empresarial não passa agora: a tarefa é estreita, já houve trabalho com prompt e RAG e o contrato é estável, mas o histórico ainda não representa suficientemente a variação real.

O atendimento ao Cliente também não passa, porém por outro motivo. Ele possui muitos dados, mas a própria tarefa é aberta e instável.

Essa comparação mostra por que o framework precisa ser aplicado pergunta por pergunta. Não existe uma resposta única para a empresa inteira nem para a categoria genérica de documento ou atendimento.

Por que a dimensão de dados recebe peso maior

A aula também antecipa que, na ponderação AHP utilizada por trás do framework, a dimensão de dados recebe maior peso do que as demais.

A justificativa não é preferência arbitrária. Um prompt melhor pode ser escrito amanhã. Um contrato pode ser redesenhado. Uma arquitetura de RAG pode ser revisada. Dado histórico representativo não aparece por decisão imediata da equipe.

Ele depende de tempo, coleta ou de uma fonte alternativa confiável.

Por isso, a insuficiência de dados possui uma característica estrutural diferente das outras limitações e recebe um peso maior na modelagem que será detalhada no próximo capítulo.

Mesmo assim, a ponderação não transforma uma condição necessária em algo compensável. O gate de governança continua anterior à análise e as perguntas vermelhas continuam tendo significado próprio.

O próximo passo do framework

No próximo capítulo, esse raciocínio deixa de ser apenas uma análise manual e passa a ser executado no terminal.

A implementação apresenta os pesos derivados pelo AHP, a razão de consistência e a recomendação para os três casos.

A análise também incorpora dimensões econômicas como NPV e breakeven, além de simulação de Monte Carlo e o valor de esperar utilizando Real Options.

A ideia é transformar o framework em uma ferramenta reproduzível. Em vez de uma equipe dizer apenas que acredita que fine-tuning vale a pena, cada caso passa por gates, perguntas, justificativas e critérios econômicos que podem ser registrados e revisitados.

Essa disciplina começa, portanto, por uma regra de decisão antes de começar o treinamento. A tecnologia só entra quando o problema, os dados, a estabilidade e a governança realmente sustentam o investimento.

NA PRÁTICA

- Compare três tarefas do seu contexto e aplique as quatro condições necessárias separadamente.
- Diferencie um caso de “ainda não” por falta de dados de um caso de “não” por tarefa aberta ou instável.
- Documente a justificativa sem permitir que um score médio compense uma condição essencial reprovada.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-2)”?
- Qual relação existe entre Condições necessárias e Gate de governança no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu apresentei o framework de decisão para avaliar quando fine-tuning realmente faz sentido.
- Vou trabalhar com três casos.
- O objetivo é aplicar as quatro perguntas com justificativa registrada e acrescentar um gate de governança que roda antes de qualquer ponderação.

Aula 3 — Quando Fazer Fine-Tuning (Decision Framework) (PT-3)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-3)”.
- Relacionar AHP e Razão de consistência às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• AHP	• Razão de consistência
• NPV	• Break-even
• Monte Carlo	• Real Options
• LoRA e QLoRA	• DPO

Conteúdo da aula

Nos capítulos anteriores, eu construí e apliquei o framework de quatro perguntas para decidir quando o fine-tuning realmente faz sentido. Agora eu tiro essa decisão do slide e coloco o ferramental para rodar no terminal.

O código executa os três casos da Amplitude Seguros, aplica o gate de governança, deriva pesos por AHP, verifica a consistência do julgamento, calcula NPV e break-even, executa dez mil simulações de Monte Carlo e, quando a reprovação é temporária por insuficiência de dados, calcula o valor de esperar usando Real Options.

A mudança é importante porque o fine-tuning não deveria ser aprovado apenas por um sinal verde informal. Quando o investimento envolve dados, treinamento, operação e manutenção, eu quero números, probabilidades e justificativas que possam ser apresentadas a um comitê e revisitadas depois.

Executando o Decision Framework no terminal

A demonstração começa com o Decision Framework Tool executado no terminal. Antes da análise, a suíte automatizada roda vinte e nove testes para validar o comportamento esperado.

Depois, o script imprime os pesos derivados, a razão de consistência, os resultados financeiros, a simulação de Monte Carlo, o valor de espera para o caso elegível a Real Options e uma sessão de comitê com pesos agregados de avaliadores hipotéticos.

O gate de governança permanece anterior a tudo isso. Ele não é uma variável compensável. Os casos só seguem para ponderação e análise financeira quando podem prosseguir do ponto de vista legal e de compliance.

AHP como formalização do julgamento

A primeira técnica é o AHP, Analytic Hierarchy Process.

Os pesos das quatro perguntas não são escolhidos diretamente como percentuais arbitrários. Eles são derivados de uma matriz de comparação pareada, em que cada critério é comparado aos demais por uma escala de importância relativa.

Na demonstração, a pergunta sobre dados suficientes recebe importância maior porque representa uma restrição que não se resolve apenas por decisão imediata da equipe. Um prompt pode ser reescrito. Um contrato pode ser redesenhado. Histórico representativo depende de tempo, coleta ou outra fonte de dados.

Da matriz saem pesos aproximados de 0,141 para as perguntas 1 e 2, 0,455 para dados suficientes e 0,263 para estabilidade.

Razão de consistência e decisão em grupo

O AHP também calcula uma razão de consistência. Isso permite detectar comparações contraditórias antes de confiar nos pesos.

Na matriz da demonstração, a razão fica em aproximadamente 0,038, abaixo do limiar de 10% utilizado como referência. O julgamento é, portanto, suficientemente coerente.

O código também simula um comitê com três perfis: produto, compliance e engenharia. Cada perfil possui sua própria matriz, e as matrizes são agregadas pela média geométrica célula a célula.

Os pesos mudam um pouco, mas os três veredictos continuam iguais: Auto aprovado, Saúde Empresarial ainda não e Atendimento ao Cliente reprovado. Isso mostra que a decisão não depende totalmente de quem está na sala.

NPV, break-even e Monte Carlo

Com os pesos definidos, o framework passa para a análise econômica.

O NPV representa o valor presente dos benefícios futuros menos o investimento necessário. O break-even mostra em que mês o retorno acumulado cobre o investimento.

Na demonstração, a taxa de desconto é de 1% ao mês, coerente com o perfil mais avesso a risco da Amplitude Seguros. Para Auto, o NPV em 24 meses fica positivo e o break-even aparece por volta do décimo mês.

Mesmo assim, eu não quero depender de um único cenário determinístico. Volume de chamadas, economia por requisição e custo operacional variam na vida real.

Por isso, o framework executa dez mil simulações de Monte Carlo. Para Amplitude Auto, a demonstração apresenta probabilidade de retorno positivo de 100% no horizonte analisado, inclusive nos cenários simulados menos favoráveis.

A utilidade da simulação é transformar premissas incertas em uma distribuição de resultados possíveis, em vez de tratar uma única planilha como verdade absoluta.

A demonstração também deixa claro que a análise financeira utiliza números ilustrativos, calibrados para o caso da Amplitude Seguros. O valor real de um projeto precisa ser recalculado com volume, custo de inferência, custo de treinamento, taxa de erro e impacto econômico do próprio contexto da empresa. A ferramenta organiza o cálculo, mas não cria premissas de negócio que não existem.

Saúde Empresarial e o valor de esperar

Saúde Empresarial é o caso mais interessante porque não recebe um simples não.

O framework reprova o treinamento agora porque o score de dados ainda não alcança o limiar necessário. Mas essa é uma condição que pode melhorar com tempo e coleta.

Por isso, o caso é elegível a Real Options. A pergunta passa a ser quanto vale manter a opção de treinar mais tarde, quando a qualidade do dataset melhorar.

O código deriva da volatilidade a partir das simulações de Monte Carlo e monta uma árvore binomial de nove passos, um por mês, até o momento estimado em que o score de dados pode cruzar o limiar.

A decisão é induzida de trás para frente. Em cada nó, o algoritmo compara treinar naquele momento com continuar esperando.

Decidir imediatamente possui valor econômico baixo porque o risco associado ao dado insuficiente supera a economia esperada. A opção de esperar, por outro lado, recebe valor positivo, na ordem de algumas centenas de reais na demonstração.

O valor varia entre execuções porque a volatilidade vem da própria simulação. O ponto não é decorar o número, mas transformar espera em decisão econômica mensurável.

Também fica registrada uma reavaliação futura. Esperar não significa abandonar o caso, mas assumir o compromisso de rodar novamente o framework quando houver mais dados.

Por que Atendimento ao Cliente não recebe Real Options

O atendimento ao cliente mostra o limite dessa técnica.

Seu score composto pode parecer relativamente alto e até superar o de Saúde Empresarial. Se eu decidisse pela média, poderia aprovar o caso incorretamente.

O framework, porém, continua avaliando condições essenciais. A tarefa não é estreita e repetida e também não é suficientemente estável. A dimensão de dados está verde.

Uma diferença importante dessa etapa é que o score ponderado não substitui os gates. Ele ajuda a quantificar importância e robustez, mas não transforma uma pergunta essencial em algo que possa ser compensado pela média. Isso explica por que Atendimento ao Cliente pode apresentar score composto superior ao de Saúde Empresarial e ainda assim ser rejeitado. O framework continua olhando para as condições necessárias que ficaram vermelhas.

Esperar mais meses não resolve esses problemas. Real Options só faz sentido quando a reprovação pode melhorar com o tempo, como insuficiência de dados. Ela não transforma uma tarefa aberta em estreita nem estabiliza uma política que muda por natureza.

Por isso, Atendimento ao Cliente é reprovado sem valor de espera e continua mais adequado a técnicas como prompt, RAG, context engineering e roteamento humano.

Aprovar fine-tuning não escolhe a técnica

A saída do framework registra outra distinção importante: aprovar fine-tuning responde se vale a pena treinar, não como treinar.

Essa segunda decisão será aprofundada nos módulos 3 e 4. Mesmo depois de um caso passar por dados, estabilidade, governança e análise financeira, ainda preciso escolher técnica, provedor e infraestrutura.

O tipo de fine-tuning depende do problema, do hardware, do orçamento e da distância entre o comportamento desejado e aquilo que o modelo base já sabe fazer.

Full Fine-Tuning, LoRA e QLoRA

Full Fine-Tuning ajusta todos os parâmetros do modelo. É mais caro e exige mais infraestrutura, mas oferece maior liberdade de adaptação quando o domínio está distante do comportamento original.

O material cita como referência histórica o Codex, derivado de treinamento especializado em código e associado ao surgimento de experiências como o GitHub Copilot.

LoRA segue outro caminho. O modelo base permanece congelado e apenas adaptadores menores são treinados. Isso reduz memória, custo e tempo de treinamento.

A aula apresenta LoRA como uma escolha muito comum em produção e cita um caso da Checkr, em classificação de registros de background check, como exemplo de redução de custo e aumento de velocidade em relação a uma solução anterior baseada em modelo maior.

QLoRA combina adaptadores com quantização, tipicamente em 4 bits, para reduzir ainda mais o consumo de memória. O exemplo do Guanaco mostra como modelos grandes puderam ser ajustados com hardware muito menor do que seria necessário em Full Fine-Tuning tradicional.

Instruction Tuning, RLHF, DPO e destilação

Instruction Tuning ensina o modelo a seguir instruções em linguagem natural em uma variedade de tarefas. O exemplo citado é o FLAN, ajustado sobre muitas tarefas para melhorar a generalização diante de novas instruções.

RLHF e DPO trabalham com preferência entre respostas. RLHF utiliza um modelo de recompensa e depois reinforcement learning. DPO simplifica o pipeline ao otimizar diretamente sobre pares preferidos e rejeitados.

A destilação busca transferir comportamento de um modelo grande para um menor, frequentemente usando exemplos produzidos pelo próprio modelo maior. O objetivo é reduzir custos de inferência mantendo capacidades relevantes.

O cheat sheet da disciplina ainda registra técnicas mais recentes, como GRPO e RFT, relacionadas a ajuste por recompensa verificável, para quem quiser aprofundar além do escopo desta aula.

O cheat sheet e a segunda decisão

O cheat sheet funciona como um mapa para escolher a técnica depois que o treinamento já foi aprovado.

Ele compara requisitos de dados, hardware, orçamento, hiperparâmetros e exemplos de mercado. Também propõe perguntas práticas: quão distante o domínio está do modelo genérico, quanto orçamento existe e qual latência a produção exige.

A lógica é a mesma do Decision Framework, mas aplicada à segunda decisão. Primeiro eu provo que fine-tuning merece existir. Depois escolho Full Fine-Tuning, LoRA, uma API gerenciada ou outra abordagem.

Essa separação evita escolher uma tecnologia de treinamento antes de provar que o investimento faz sentido.

Risco operacional de provedor

A aula também introduz o risco operacional do provedor.

Um serviço self-service pode mudar de estratégia, remover uma funcionalidade ou deixar de oferecer determinado fluxo de fine-tuning. O material registra mudanças observadas no mercado durante a preparação da disciplina como exemplo desse risco.

Isso não muda se fine-tuning vale a pena. Muda onde treinar.

O material ainda chama atenção para mudanças concretas em ofertas self-service de fine-tuning e para a possibilidade de um provedor abandonar um produto. A consequência prática é que a decisão técnica precisa vir acompanhada de uma estratégia de saída. Se o treinamento depende de uma única API sem portabilidade, o risco operacional pode se tornar maior do que o ganho inicial de conveniência.

Por isso, arquitetura e planejamento precisam considerar dependência de fornecedor, portabilidade de artefatos e alternativas locais ou em outros provedores.

O modelo de fronteira pode alcançar o modelo customizado

Existe ainda outra forma de obsolescência. Um modelo de fronteira futuro pode superar um modelo especializado que hoje representa vantagem.

O material utiliza um caso do setor jurídico para mostrar essa evolução. Um modelo customizado pode ser superior em determinado momento e, algum tempo depois, modelos generalistas mais novos podem alcançar ou ultrapassar seu desempenho.

Isso não significa que o treinamento anterior tenha sido um erro. Ele pode ter gerado valor durante o período em que possuía vantagem.

Significa que fine-tuning tem prazo de validade operacional e precisa ser reavaliado contra novas baselines, novos custos e novos modelos base.

Missão prática do módulo

O módulo termina com uma missão prática em quatro partes.

Primeiro, eu aplico as quatro perguntas a um caso real do meu próprio contexto e registro justificativa para cada resposta.

Segundo, pondero as perguntas. Posso utilizar o AHP completo ou um score mais simples, desde que documento por que determinado critério pesa mais.

Terceiro, estimo retorno financeiro. Não é obrigatório reproduzir toda a simulação de Monte Carlo. Um NPV determinístico, uma economia recorrente ou um break-even já tornam o raciocínio econômico explícito.

Se a única reprovação for dada como insuficiente, documento também quantos meses seriam necessários para acumular dados e por que esperar pode valer mais do que treinar agora. Se a reprovação vier de tarefa aberta ou instável, registro porque Real Options não se aplica.

Por fim, adapto o Decision Framework Tool para os números do meu caso e executo a recomendação.

Concluir que a tarefa não precisa de fine-tuning também é uma entrega válida. O objetivo é decidir corretamente, não decidir treinar sempre.

No próximo módulo, os casos elegíveis deixam a fase de decisão e entram na preparação de dados, transformando documentos brutos da Amplitude Seguros em datasets JSONL limpos e balanceados para treinamento.

NA PRÁTICA

- Adapte o Decision Framework aos números de um caso real ou plausível do seu contexto.
- Registre a ponderação dos critérios e uma análise econômica simples, como NPV, economia recorrente ou break-even.
- Se a única reprovação for insuficiência de dados, registre quando o caso deverá ser reavaliado e por que esperar pode ter valor.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-3)”?
- Qual relação existe entre AHP e Razão de consistência no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

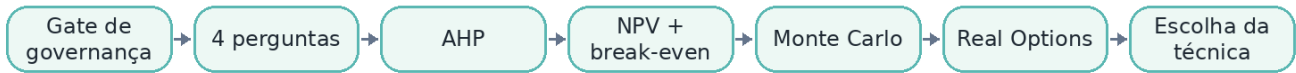
- Nos capítulos anteriores, eu construí e apliquei o framework de quatro perguntas para decidir quando o fine-tuning realmente faz sentido.
- O código executa os três casos da Amplitude Seguros, aplica o gate de governança, deriva pesos por AHP, verifica a consistência do julgamento, calcula NPV e break-even, executa dez mil simulações de Monte Carlo e, quando a reprovação é temporária por insuficiência de dados, calcula o valor de esperar usando Real Options.
- A mudança é importante porque o fine-tuning não deveria ser aprovado apenas por um sinal verde informal.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 1

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-1\)](#)
- [Revisar a Aula 2: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-2\)](#)
- [Revisar a Aula 3: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-3\)](#)

[↑ Voltar ao mapa da disciplina](#)

Unidade 2

Preparação de Datasets para Fine-Tuning

Preparação de dados de treinamento com gates de relevância, OCR, parsing, validação, redução de PII, representação canônica, deduplicação e balanceamento para aumentar qualidade e diversidade.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Preparação de Datasets para Fine-Tuning (PT-1)
2	Preparação de Datasets para Fine-Tuning (PT-2)

Aula 1 — Preparação de Datasets para Fine-Tuning (PT-1)

Unidade 2 • Preparação de Datasets para Fine-Tuning

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Preparação de Datasets para Fine-Tuning (PT-1)”.
- Relacionar OCR e Tesseract às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• OCR	• Tesseract
• Parser tolerante	• Validação de esquema
• PII scrubbing	• JSONL canônico
• Rastreabilidade	

Conteúdo da aula

O módulo anterior terminou com uma decisão importante. Para Amplitude Auto, o framework indicou que fine-tuning já fazia sentido. Para a Amplitude Saúde Empresarial, a recomendação foi esperar, principalmente porque ainda faltava dado suficiente. Só que decidir treinar ou decidir esperar não produz automaticamente um dataset. A partir daqui, eu começo o trabalho de transformar documentos reais em exemplos estruturados que possam ser utilizados mais adiante no treinamento.

O ponto de partida são documentos de apoio que chegam de fora da plataforma da Amplitude Seguros, como orçamentos de oficina e recibos médicos. Esses documentos podem existir como imagens PNG, fotos de papel ou digitalizações. Antes de limpar ou estruturar qualquer coisa, eu preciso extrair o texto. Por isso, o primeiro componente técnico deste pipeline é OCR, reconhecimento óptico de caracteres.

Mesmo Saúde Empresarial ainda não estando pronto para treinamento, faz sentido preparar os dados com o mesmo rigor. A espera recomendada no módulo 1.3 não é um período de inatividade. É justamente o período em que eu posso melhorar coleta, qualidade, governança e cobertura de variação até o caso cruzar o limiar exigido pelo framework.

Antes do OCR, o documento precisa merecer entrar no dataset

O primeiro erro que eu quero evitar é coletar tudo que existe no arquivo da empresa e chamar isso de dataset.

Preparar dados para fine-tuning não é acumular documentos. Eu preciso definir critérios de relevância e qualidade antes da ingestão.

Esse raciocínio se aproxima da ideia de data-centric AI: melhorar sistematicamente o dado e estabelecer critérios explícitos para aquilo que entra no treinamento, em vez de assumir que mais volume sempre significa mais qualidade.

A aula cita o trabalho LIMA como exemplo dessa lógica. Um conjunto pequeno, mas cuidadosamente curado, pode produzir resultados melhores do que volumes muito maiores de exemplos menos controlados. O número de exemplos importa, mas a qualidade e a representatividade importam antes.

O gate de relevância com quatro perguntas

Antes de coletar qualquer documento, eu aplico um gate com quatro perguntas.

Primeiro, o documento contém o ground truth da tarefa? Eu preciso conseguir observar uma entrada real e uma resposta correta.

Segundo, o documento vem do fluxo real de produção e não foi inventado apenas para ilustrar o problema?

Terceiro, ele ajuda a cobrir a variação real de formatos que a tarefa apresenta?

Quarto, ele pode ser utilizado em treinamento do ponto de vista de sensibilidade e compliance?

As quatro respostas precisam ser verdadeiras. Se uma falha, o documento não entra. Esse gate acontece antes de OCR e parsing. Ele decide se vale a pena gastar qualquer esforço de preparação com aquele item.

Aplicando o gate aos documentos da Amplitude Seguros

A demonstração aplica o gate a sete candidatos plausíveis.

No caso de Auto, eu considero orçamento de oficina, boletim de ocorrência, fotografia do veículo danificado e transcrição da ligação de abertura do sinistro.

Em Saúde Empresarial, aparecem recibo médico, prontuário médico completo e cadastro de beneficiários.

Dos sete candidatos, apenas dois passam: orçamento de oficina e recibo médico.

O boletim de ocorrência não entrega segurado, placa e valor de maneira suficientemente confiável para funcionar como gabarito da tarefa. A foto do veículo mostra o dano, mas não contém o documento textual com os campos desejados. A transcrição da ligação pode conter os dados, mas a estrutura varia demais entre atendentes para funcionar como exemplo consistente de extração.

No caso de Saúde Empresarial, o prontuário completo passa nos critérios técnicos, mas falha em compliance porque contém muito mais dados clínicos do que o necessário para extrair beneficiário, procedimento e valor. O princípio de necessidade da LGPD exige minimizar o dado utilizado para a finalidade.

O cadastro de beneficiários falha por outra razão. Ele é uma tabela de referência útil ao processo de negócio, mas não representa um par de entrada e saída da tarefa de extração. Dado real e limpo não é automaticamente dado adequado para treinar o extrator.

Do documento bruto ao exemplo validado

Depois do gate de relevância, eu organizo o pipeline de ponta a ponta.

O fluxo começa no documento bruto, passa por OCR, segue para um parser tolerante, entra em validação de esquema e termina em um exemplo estruturado pronto para compor o dataset.

Cada estágio resolve um problema diferente. O OCR transforma a imagem em texto. O parser interpreta variações de rótulo e localiza os campos. A validação de esquema verifica se o resultado faz sentido. O exemplo estruturado preserva entrada, saída e metadados para as etapas seguintes.

Misturar tudo em uma única função tornaria mais difícil descobrir onde um erro nasceu. Separar as etapas mantém rastreabilidade.

OCR com Tesseract e idioma português

O motor utilizado na demonstração é o Tesseract.

Ele é um OCR open source com uma longa história de desenvolvimento e utiliza um mecanismo baseado em rede neural nas versões modernas.

Para documentos em português, eu preciso instalar o pacote de idioma correspondente. Isso é importante porque nomes próprios, acentuação, cedilha e termos médicos podem ser reconhecidos de forma inadequada quando o motor opera apenas com configuração de inglês.

O OCR entra especificamente nos documentos de apoio. O formulário principal de sinistro pode chegar estruturado pelo aplicativo ou portal da própria Amplitude. Já o orçamento produzido por uma oficina ou o recibo emitido por uma clínica nasce fora desse sistema e não segue um padrão controlado pela seguradora.

Por que o parser precisa ser tolerante

O texto extraído pelo OCR ainda não resolve o problema.

Cada oficina pode escrever o mesmo campo de maneira diferente. Um documento usa "segurado", outro usa "nome do segurado". Um usa "placa", outro "placa do veículo". Em Saúde Empresarial, um recibo escreve "beneficiário", outro "paciente beneficiário". Um usa "valor", outro "valor cobrado".

Se o parser procurar apenas um rótulo fixo, funcionará em uma fonte e quebrará na próxima.

Por isso, a implementação utiliza expressões regulares tolerantes a pequenas variações de rótulo e não depende de uma posição fixa no texto.

Essa flexibilidade é importante porque o documento de amanhã pode vir de uma oficina que nunca apareceu durante o desenvolvimento. Um parser de demonstração reconhece um layout conhecido. Um parser de produção precisa tolerar variações plausíveis sem começar a inventar campo quando não encontrou o valor.

O risco de aceitar extração incompleta

Sem validação de esquema, uma falha de parsing pode entrar silenciosamente no dataset.

Se o parser procura apenas "segurado" e encontra "nome do segurado", ele pode devolver o campo vazio. Se eu aceitar esse exemplo mesmo assim, o modelo passa a aprender que, em alguns documentos, não identificar o segurado é uma saída normal.

Multiplicado por centenas de documentos, isso deixa de ser um erro isolado e vira ruído sistemático.

A regra, portanto, é rejeitar o exemplo quando campos obrigatórios estão ausentes ou quando o valor extraído é inválido. Quase certo não é suficiente para dado de treinamento.

Executando OCR, parsing e validação de verdade

Na demonstração, o pipeline roda com o binário real do Tesseract sobre quatro imagens sintéticas de documentos.

São dois orçamentos de oficina e dois recibos médicos, cada um com pequenas diferenças de layout e rótulos.

O texto bruto do OCR não é perfeito. Existem espaços extras e quebras de linha pouco elegantes. Isso é esperado e, por isso, eu preservo esse texto como entrada do exemplo em vez de tentar embelezá-lo manualmente.

O parser fica separado e extrai apenas os campos necessários.

Em um dos orçamentos, o OCR identifica segurado, placa do veículo e valor total do reparo. O parser converte o valor monetário brasileiro para número decimal e devolve os campos esperados.

Em um recibo com layout diferente, o parser reconhece "paciente beneficiário" e "valor cobrado" sem depender do rótulo usado no primeiro documento.

Confiança do OCR como metadado

Junto com o texto, eu também guardo a confiança média reportada pelo Tesseract.

Nos quatro documentos da demonstração, ela fica aproximadamente entre 94% e 96%.

Esse número não é inventado pelo pipeline. Ele vem da confiança calculada pelo OCR palavra a palavra e depois agregada.

Em um documento de digitalização ruim, a confiança tenderia a cair. Essa informação pode ser usada mais adiante para decidir se um exemplo precisa de revisão humana antes de entrar no treinamento.

No módulo seguinte, essa confiança se transforma em gate operacional. Aqui ela já entra no metadado para não ser perdida.

Testes automatizados e caminho de rejeição

Antes de confiar no dataset gerado, o pipeline executa vinte e oito testes automatizados.

Os primeiros isolam a conversão de valores em reais para garantir que a vírgula decimal brasileira não seja interpretada de maneira errada.

A maior parte dos testes roda contra os quatro documentos. O OCR precisa extrair texto real, o exemplo precisa passar pela validação, os campos precisam bater com os valores esperados e a confiança precisa permanecer em uma faixa válida.

Os testes finais verificam o caminho contrário. Um exemplo sem placa é rejeitado. Um exemplo com valor zero também é rejeitado.

Eu não quero apenas provar que o pipeline aceita o correto. Preciso provar que ele recusa o incorreto.

O esquema canônico de quatro campos

Todo exemplo aceito passa a carregar quatro campos em uma ordem conceitual estável.

O primeiro é instrução. Ela descreve a tarefa em linguagem natural e permanece a mesma dentro de um caso.

O segundo é entrada. Aqui eu guardo o texto bruto produzido pelo OCR, sem edição manual.

O terceiro é saída. É o objeto estruturado produzido pelo parser. Em Auto, contém segurado, placa e valor. Em Saúde Empresarial, beneficiário, procedimento e valor.

O quarto é metadata. Ele registra o caso, o arquivo de origem, a confiança de OCR e o resultado das validações.

Esse esquema é deliberadamente canônico. Ele não é ainda o formato específico de Vertex AI nem o formato usado pelo treinamento local com MLX. A adaptação para cada provedor fica para os módulos de treinamento.

Por que manter uma representação canônica

Separar preparação de formatação por provedor evita acoplamento desnecessário.

Se amanhã eu trocar o Vertex AI por outro serviço ou migrar parte do treinamento para LoRA local, não quero repetir OCR, parsing, compliance e validação.

Eu mantenho um dataset intermediário estável e crio conversores específicos para cada destino.

Esse desenho também facilita testes. O pipeline de qualidade pode ser validado sem depender da API do provedor e sem exigir que o formato de armazenamento de um fornecedor invada a lógica de preparação.

Rastreabilidade do texto bruto

Guardar o texto OCR bruto possui duas vantagens importantes.

A primeira é rastreabilidade técnica. Se um exemplo apresentar comportamento estranho, eu consigo voltar ao texto original e identificar se o problema nasceu no documento, no OCR ou no parser.

A segunda é auditoria. No caso de Saúde Empresarial, dado de saúde é categoria sensível. Manter a origem e o caminho de transformação ajuda a registrar as operações de tratamento e a entender de onde cada exemplo surgiu.

Existe, porém, uma tensão. Preservar o texto bruto com nome completo significa também preservar PII em um material que pode ser usado para treinamento. Em produção, isso exige uma etapa adicional de pseudonimização antes que o dado chegue ao modelo.

PII scrubbing antes do treinamento

A aula transforma essa recomendação em um gate executável de PII scrubbing.

O pipeline trata nome e CPF nos casos de Auto e Saúde Empresarial. O CPF é validado pelo dígito verificador, não apenas pelo formato textual.

Os nomes são identificados quando aparecem ancorados por rótulos conhecidos, como segurado ou beneficiário.

Depois da redação, nome e CPF deixam de aparecer no exemplo de treino. Placa e valor permanecem, porque são necessários para a tarefa e, isoladamente naquele contexto, não são tratados da mesma forma que o identificador direto.

O limite dessa abordagem é assumido explicitamente. Um nome solto em texto livre não seria capturado apenas por âncora de rótulo. Para esse cenário, eu precisaria de reconhecimento de entidades nomeadas mais robustas, semelhante ao tipo de estratégia utilizada por ferramentas como Microsoft Presidio.

O objetivo não é prometer anonimização completa. É reduzir PII de maneira verificável e deixar claro onde o método deixa de ser suficiente.

Validação continua sendo o portão final

Mesmo depois de OCR, parsing e PII scrubbing, o exemplo ainda precisa passar pela validação.

Campo obrigatório ausente é motivo de rejeição. Valor zero ou negativo também é rejeitado porque normalmente indica que o parser não conseguiu encontrar corretamente o número.

Nos quatro documentos usados na aula, todos passam. Isso não significa que o pipeline foi desenhado para aceitar tudo. Ele foi desenhado para aceitar apenas quando as condições mínimas são atendidas.

É mais barato rejeitar um exemplo ruim durante a preparação do que descobrir o problema depois de treinar o modelo sobre ele.

Quando quatro exemplos viram centenas

Até aqui, eu trabalhei documento por documento. Essa escala é útil para provar que extração e validação funcionam, mas um dataset de fine-tuning real contém centenas ou milhares de exemplos.

Quando a escala aumenta, surgem problemas que não aparecem em quatro documentos.

Um deles é viés de amostragem. Se 90% dos orçamentos vierem de poucas oficinas parceiras, o modelo pode aprender muito bem aqueles formatos e generalizar mal para oficinas novas.

Outro é a repetição quase idêntica. Duas unidades da mesma rede podem utilizar o mesmo sistema e gerar documentos com estrutura quase igual, mudando apenas nome e valor. Um dataset cheio de quase-duplicatas aumenta artificialmente a sensação de diversidade e desperdiça capacidade de treinamento repetindo o mesmo padrão.

Esses problemas não são de extração individual. São problemas de composição do dataset como conjunto.

Preparando o próximo passo

Esta aula fecha a transformação de documento em exemplo.

Eu começo filtrando relevância, extraio texto com OCR, interpreto variações com parser tolerante, valido o contrato, preservo metadados, reduzo PII e escrevo o resultado em JSONL canônico.

No próximo capítulo, a unidade de análise muda. Em vez de perguntar se um exemplo individual está correto, eu preciso perguntar se o conjunto inteiro está repetitivo, enviesado ou desequilibrado.

É aí que entram limpeza, deduplicação e balanceamento. O objetivo passa a ser transformar muitos exemplos individualmente válidos em um dataset que cubra a variação real da tarefa e ensine padrão, não repetição.

NA PRÁTICA

- Aplique o gate de relevância a pelo menos quatro candidatos de fonte de dados e mantenha ao menos um rejeitado com justificativa.
- Defina um esquema canônico com instrução, entrada, saída e metadados.
- Descreva onde OCR, parsing, validação e PII scrubbing entram no pipeline e qual falha cada etapa precisa impedir.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Preparação de Datasets para Fine-Tuning (PT-1)”?
- Qual relação existe entre OCR e Tesseract no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- O módulo anterior terminou com uma decisão importante.
- O ponto de partida são documentos de apoio que chegam de fora da plataforma da Amplitude Seguros, como orçamentos de oficina e recibos médicos.
- Mesmo Saúde Empresarial ainda não estando pronto para treinamento, faz sentido preparar os dados com o mesmo rigor.

Aula 2 — Preparação de Datasets para Fine-Tuning (PT-2)

Unidade 2 • Preparação de Datasets para Fine-Tuning

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Preparação de Datasets para Fine-Tuning (PT-2)”.
- Relacionar MinHash e LSH às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• MinHash	• LSH
• Deduplicação	• Recall
• Balanceamento por temperatura	• Entropia de Shannon
• Diversidade	

Conteúdo da aula

No capítulo anterior, eu transformei documentos individuais em exemplos estruturados. O pipeline já consegue decidir se um documento é relevante, extrair texto com OCR, interpretar os campos com um parser tolerante, validar o esquema, registrar metadados e reduzir PII antes de escrever o exemplo no dataset. Isso prova que a unidade individual funciona. Ainda não prova que o dataset como conjunto está bom.

Quando eu saio de quatro exemplos e começo a trabalhar com dezenas, centenas ou milhares de registros, surgem problemas que não aparecem na validação documento por documento. Dois deles são especialmente importantes: quase-duplicatas e desbalanceamento de fonte.

Um dataset pode ser formado apenas por exemplos individualmente corretos e, mesmo assim, induzir o modelo ao erro. Se muitos registros repetem praticamente o mesmo padrão, o treinamento reforça informação redundante. Se uma única oficina ou clínica domina a distribuição, o modelo aprende muito bem aquele formato e generaliza pior para fontes novas.

Por isso, nesta etapa eu mudo a unidade de análise. Não quero apenas perguntar se cada exemplo é válido. Quero saber se o conjunto inteiro possui diversidade suficiente para ensinar padrão em vez de repetição.

Escala revela problemas que quatro exemplos escondem

Imagine que a Amplitude Seguros processa centenas de sinistros por mês. Uma oficina parceira de grande volume pode enviar orçamentos toda semana, sempre gerados pelo mesmo sistema e quase com o mesmo layout.

Se essa oficina representar mais da metade do dataset, o modelo verá aquele formato continuamente. Ele pode ficar excelente para documentos dessa fonte e mais frágil quando receber o orçamento de uma oficina nova.

O problema é semelhante em Saúde Empresarial. Uma clínica que processa muitos reembolsos pode dominar o conjunto e fazer com que layouts menos frequentes quase desapareçam do treinamento.

É exatamente nesse ponto que quantidade deixa de ser sinônimo de cobertura. Um dataset grande pode representar apenas a repetição de poucas fontes.

Quase-duplicatas

O primeiro problema é a quase-duplicata.

Um mesmo sinistro pode aparecer duas vezes: porque o operador reenviou um documento ou porque houve uma nova digitalização. O OCR pode produzir pequenas diferenças entre as duas versões, como espaço extra, uma quebra de linha diferente ou um caractere reconhecido de outra forma.

Para uma comparação visual rápida, os registros parecem diferentes. Para o treinamento, eles carregam praticamente a mesma informação.

Se eu mantiver as duas cópias, o modelo recebe um reforço artificial daquele padrão. Ele não aprende uma nova variação do problema. Apenas vê de novo algo que já estava no conjunto.

Em escala, essa repetição desperdiça computação e pode aumentar o risco de memorização.

Por que deduplicação muda o modelo, não apenas o arquivo

A aula usa um estudo de pesquisadores do Google Research sobre deduplicação de dados de treinamento para mostrar que isso não é apenas limpeza estética.

Ao analisar o corpus C4, os pesquisadores encontraram casos extremos de repetição, incluindo uma mesma frase aparecendo dezenas de milhares de vezes.

O resultado da deduplicação aparece no comportamento do modelo. O estudo citado mostra redução forte na emissão de texto memorizado palavra por palavra, necessidade de menos passos de treinamento para atingir desempenho equivalente ou melhor e redução de contaminação entre treino e teste.

Esse último ponto é especialmente importante. Quando exemplos iguais ou quase iguais aparecem nos dois conjuntos, a avaliação passa a medir parcialmente a memória, não a generalização. A métrica parece melhor do que o modelo realmente é.

Portanto, deduplicar protege treinamento, custo e avaliação ao mesmo tempo.

Desbalanceamento de fonte

O segundo problema é a concentração de exemplos em poucas fontes.

Uma oficina ou clínica pode produzir mais documentos simplesmente porque possui maior volume de atendimento. Isso é uma característica operacional, mas não significa que ela deva receber o mesmo peso proporcional no dataset de fine-tuning.

Se uma fonte representa 60% do conjunto, ela também representa 60% das oportunidades de ajuste durante o treinamento. O modelo passa a receber muito mais evidência sobre aquele layout do que sobre os demais.

O risco aparece justamente em produção, quando chega uma fonte nova ou pouco frequente. O modelo foi treinado para um retrato enviesado da distribuição histórica e não para a variedade que eu gostaria que ele suportasse.

Padronização antes da comparação

Antes de procurar duplicatas, eu faço uma normalização básica do texto.

A comparação não deveria tratar letras maiúsculas, espaços duplicados ou sobras no início e no fim como diferença de conteúdo.

Por isso, o texto é padronizado com transformação para minúsculas, colapso de espaços e remoção de excesso nas extremidades.

Essa etapa não tenta reescrever semanticamente o documento. Ela apenas elimina diferenças cosméticas que poderiam fazer duas versões praticamente iguais parecerem mais distantes do que realmente são.

MinHash para aproximar similaridade

Comparar cada documento contra todos os outros funciona em conjuntos pequenos, mas cresce rapidamente com o número de exemplos.

A estratégia utilizada na aula é transformar cada texto em uma assinatura MinHash.

O texto é representado por pequenos fragmentos e a assinatura resume esse conjunto em uma quantidade fixa de valores. A fração de posições iguais entre duas assinaturas aproxima a similaridade de Jaccard sem exigir comparar todos os fragmentos diretamente.

Na demonstração, cada exemplo recebe uma assinatura com 32 valores.

MinHash não decide sozinho se dois documentos são duplicados. Ele cria uma representação barata para reduzir o espaço de comparação.

LSH para gerar candidatos

Depois do MinHash, eu aplico LSH, Locality Sensitive Hashing.

A assinatura é dividida em bandas. Na demonstração, são oito bandas com quatro valores em cada uma.

Se dois exemplos colidem em pelo menos uma banda, eles viram candidatos a quase-duplicata.

A vantagem é que eu não preciso calcular a similaridade exata para todos os pares possíveis. O LSH reduz drasticamente o número de comparações que chegam à etapa mais cara.

No conjunto da aula, 549 comparações possíveis por força bruta são reduzidas para apenas 20 candidatos. Isso representa uma redução de aproximadamente 96,4% no trabalho de comparação.

Mesmo assim, os três pares de quase-duplicatas plantados propositalmente são encontrados. Nenhum escapa.

Refino exato antes de remover

LSH serve para localizar candidatos, não para tomar a decisão final.

Os pares encontrados passam por uma comparação exata de similaridade antes de qualquer remoção.

Essa etapa evita tratar como duplicata dois documentos que utilizam o mesmo template, mas representam casos realmente diferentes.

A aula mostra justamente esse cenário com dois orçamentos da mesma oficina. A estrutura é parecida, mas segurado, valor e demais informações pertencem a sinistros distintos.

Mesmo template não é duplicata.

A combinação entre MinHash, LSH e refino exato permite reduzir custos sem transformar proximidade visual em decisão automática.

Recall na deduplicação

Uma deduplicação eficiente não é apenas aquela que reduz comparações. Ela precisa continuar encontrando os duplicados relevantes.

Na demonstração, os três pares plantados são recuperados. Isso representa um recall perfeito para o conjunto de testes.

Esse indicador é importante porque uma otimização que economiza computação mas deixa muitas duplicatas escaparem não resolve o problema.

O objetivo é diminuir o número de pares analisados mantendo a capacidade de encontrar os casos que realmente precisam ser removidos.

Balanceamento por temperatura

Depois da deduplicação, eu ainda preciso corrigir a concentração por fonte.

A abordagem utilizada é a amostragem por temperatura. Em vez de cortar arbitrariamente qualquer fonte que passe de determinado percentual, eu atribuo pesos de acordo com a frequência original elevada a um expoente alfa.

Com alfa igual a 1, a distribuição permanece proporcional ao volume original. À medida que alfa diminui, a distribuição é suavizada em direção a uma participação mais uniforme entre as fontes.

Na demonstração, utilizo alfa igual a 0,3, valor também associado a estratégias de balanceamento em trabalhos como mT5.

O objetivo é reduzir a dominância das fontes grandes sem fingir que as fontes pequenas possuem exemplos que não existem.

Do peso contínuo à quantidade inteira

O cálculo por temperatura produz pesos contínuos. O dataset, porém, precisa terminar com uma quantidade inteira de exemplos por fonte.

Para fazer essa conversão, eu utilizo o método do maior resto, com uma restrição adicional de capacidade.

Nenhuma fonte pode receber uma quantidade maior do que o número de exemplos que realmente possui.

Essa restrição é fundamental. Balancear não significa duplicar artificialmente um exemplo minoritário apenas para preencher uma cota. Significa utilizar de forma mais equilibrada o dado disponível.

As fontes dominantes perdem participação relativa e as menores mantêm ou ganham espaço dentro do limite real de seus registros.

O efeito do balanceamento na Amplitude Auto e Saúde Empresarial

Antes do balanceamento, a Oficina Estrela representa aproximadamente 53,8% dos exemplos de Auto.

Depois da amostragem por temperatura, sua participação cai para 40%.

Em Saúde Empresarial, a Clínica Vitalis começa com aproximadamente 61,1% do conjunto e termina em 50%.

A intenção não é eliminar essas fontes. Elas continuam importantes porque representam volume real. O que eu evito é permitir que o volume de uma única origem substitua a diversidade do problema.

As fontes menores passam a ter participação suficiente para que seus formatos também influenciem o treinamento.

Entropia de Shannon como medida de diversidade

Dizer que o dataset ficou mais diverso ainda seria uma afirmação subjetiva se eu não tivesse uma medida.

Para isso, utilizo a entropia de Shannon sobre a distribuição das fontes.

Quanto mais concentrado o dataset em poucas fontes, menor a entropia. Quanto mais equilibrada a distribuição, maior o valor.

A aula também transforma essa entropia em número efetivo de fontes usando sua exponencial. Isso responde a uma pergunta mais intuitiva: a quantas fontes igualmente representadas essa distribuição equivale?

Em Auto, com quatro fontes reais, o número efetivo sobe de aproximadamente 3,279 para 3,742 depois do balanceamento.

Em Saúde Empresarial, com três fontes reais, o número efetivo sobe de aproximadamente 2,544 para 2,814.

Agora eu consigo demonstrar numericamente que o conjunto final está menos concentrado.

O dataset fica menor e melhor distribuído

A demonstração começa com 47 exemplos simulados, sendo 28 de Auto e 19 de Saúde Empresarial.

Depois da deduplicação, restam 44.

Após o balanceamento por temperatura, o dataset final fica com 34 exemplos.

O resultado é um arquivo menor, mas com menos repetição e distribuição mais equilibrada entre as fontes.

Esse é um exemplo concreto de por que remover exemplos pode melhorar um dataset. O objetivo não é maximizar a quantidade bruta. É maximizar informação útil e cobertura da tarefa.

Os testes automatizados do pipeline

Antes de confiar nessa transformação, o pipeline executa dezesseis testes automatizados.

Eles verificam a aproximação do MinHash em relação à similaridade exata, o recall do LSH, a detecção das duplicatas plantadas, a restrição de capacidade no balanceamento e o comportamento das métricas de diversidade.

Um dos princípios testados é que suavizar a distribuição não deve reduzir a diversidade medida.

Também verifico que nenhuma fonte recebe mais exemplos do que realmente possui.

Esses testes transformam limpeza e balanceamento em comportamento verificável, não em uma sequência de decisões manuais difíceis de reproduzir.

Por que implementar MinHash e LSH diretamente

A aula também registra uma decisão prática da implementação.

No momento da construção do material, não havia uma biblioteca JavaScript madura que resolvesse de forma satisfatória toda a combinação de MinHash e LSH Banding necessária para o exemplo.

Por isso, a dupla é implementada diretamente no projeto.

Isso não significa que a técnica seja didática ou exclusiva desta disciplina. MinHash e LSH aparecem em pipelines reais de deduplicação de grandes corpora.

O ponto é que a disponibilidade de biblioteca depende do ecossistema. Quando a abstração pronta não existe, eu ainda preciso compreender a técnica para implementar ou validar a solução.

Qualidade e volume precisam ser medidos juntos

O aprendizado deste módulo não é escolher qualidade no lugar de volume.

Fine-tuning continua precisando de cobertura suficiente. O que eu não posso fazer é contar exemplos repetidos ou concentrados como se fossem informação nova.

O material volta ao exemplo do trabalho LIMA para reforçar essa ideia: curadoria pode permitir que um conjunto menor seja mais útil do que um conjunto muito maior e menos controlado.

MinHash, LSH, amostragem por temperatura e entropia fornecem mecanismos formais para transformar um conjunto bruto em um dataset mais representativo.

Eu continuo medindo volume, mas agora também meço redundância e diversidade.

Missão prática do módulo

A missão prática leva esse pipeline para o contexto de cada aluno.

Primeiro, eu aplico o gate de relevância do capítulo anterior a pelo menos quatro candidatos de fonte de dados, mantendo pelo menos um candidato rejeitado com justificativa.

Segundo, declaro um esquema canônico com instrução, entrada, saída e metadata, adaptando os campos de saída para a tarefa escolhida.

Terceiro, monto ou simulo de forma realista um dataset com pelo menos quinze exemplos vindos de pelo menos três fontes diferentes.

Depois, executo deduplicação com MinHash e LSH e balanceamento por temperatura, adaptando a ferramenta disponibilizada no módulo.

Não é necessário reimplementar tudo do zero. O importante é compreender o papel de cada etapa e ter código executável com pelo menos um teste automatizado provando que o comportamento esperado foi detectado.

Também é válido chegar à conclusão de que a principal fonte disponível ainda não possui volume suficiente para treinamento. O objetivo continua sendo decidir e preparar corretamente, não forçar fine-tuning em qualquer cenário.

Da preparação para o treinamento gerenciado

Com esta etapa, o módulo 2 fecha o pipeline de preparação.

No primeiro capítulo, eu garanti que cada documento relevante poderia virar um exemplo individual válido, rastreável e com tratamento de PII.

Agora eu garanto que o conjunto não esteja dominado por quase-duplicatas nem por poucas fontes.

A partir do próximo módulo, essa mesma lógica será escalada para o volume utilizado no treinamento real da Amplitude Seguros.

É então que entram conversão para o formato do provedor, upload, acompanhamento de jobs, hiperparâmetros e monitoramento em uma API comercial gerenciada.

O treinamento só começa depois que o dado passou por esse conjunto de gates. Primeiro eu provo que o exemplo é correto. Depois provo que o dataset é diverso o suficiente. Só então vale pagar para o modelo aprender com ele.

NA PRÁTICA

- Monte ou simule um dataset com exemplos provenientes de fontes diferentes.
- Aplique deduplicação com MinHash/LSH e depois balanceamento por temperatura, respeitando a capacidade real de cada fonte.
- Compare a distribuição antes e depois usando uma medida de diversidade, como a entropia de Shannon apresentada na aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Preparação de Datasets para Fine-Tuning (PT-2)”?
- Qual relação existe entre MinHash e LSH no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

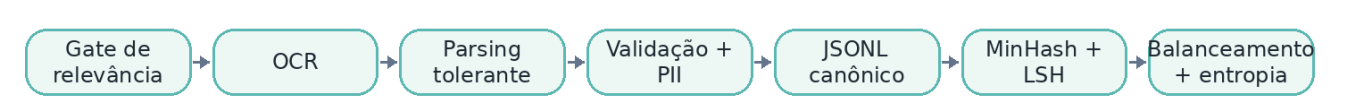
- No capítulo anterior, eu transformei documentos individuais em exemplos estruturados.
- Quando eu saio de quatro exemplos e começo a trabalhar com dezenas, centenas ou milhares de registros, surgem problemas que não aparecem na validação documento por documento.
- Um dataset pode ser formado apenas por exemplos individualmente corretos e, mesmo assim, induzir o modelo ao erro.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 2

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Preparação de Datasets para Fine-Tuning \(PT-1\)](#)
- [Revisar a Aula 2: Preparação de Datasets para Fine-Tuning \(PT-2\)](#)

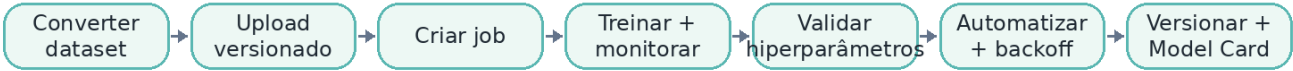
[↑ Voltar ao mapa da disciplina](#)

Unidade 3

Fine-Tuning via API (upload e train)

Treinamento gerenciado via API: conversão e upload de dados, criação e monitoramento de jobs, hiperparâmetros, automação segura, rastreabilidade, versionamento e Model Card.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Fine-Tuning via API (upload e train) (PT-1)
2	Fine-Tuning via API (upload e train) (PT-2)
3	Fine-Tuning via API (upload e train) (PT-3)
4	Fine-Tuning via API (upload e train) (PT-4)
5	Fine-Tuning via API (upload e train) (PT-5)

Aula 1 — Fine-Tuning via API (upload e train) (PT-1)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-1)”.
- Relacionar Fine-tuning gerenciado e Vertex AI às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Fine-tuning gerenciado	• Vertex AI
• Conversão de dataset	• Upload
• Job de tuning	• Monitoramento
• Artefato versionado	

Conteúdo da aula

No módulo anterior, eu fechei a preparação do dataset da Amplitude Seguros. Primeiro, filtrei quais documentos realmente mereciam virar exemplos de treino, passei por OCR, parsing tolerante, validação de esquema e PII scrubbing. Depois, olhando o conjunto como um todo, apliquei deduplicação com MinHash e LSH, balanceamento por temperatura e métricas de diversidade com entropia de Shannon. Nesse ponto, o dado já não é apenas um conjunto de documentos convertidos para JSONL. Ele passou por gates de qualidade e por uma curadoria formal antes de chegar ao treinamento.

Seria natural começar este capítulo enviando imediatamente esse dataset para algum provedor. Eu ainda não faço isso de imediato porque existe uma decisão arquitetural anterior: o provedor que parecia óbvio quando a disciplina foi planejada pode não continuar disponível quando o pipeline chegar à produção.

Essa mudança de perspectiva é importante porque o dataset pronto não significa infraestrutura de treinamento garantida. Fine-tuning via API comercial gerenciada depende de um terceiro, e esse terceiro pode alterar produto, modelo, política de acesso ou estratégia comercial independentemente da utilidade que aquela API tenha para o meu sistema.

Fine-tuning via API como segunda decisão

No módulo 1, o Decision Framework respondeu se fine-tuning valia a pena. Essa era a primeira decisão.

No módulo 2, eu tratei a condição que sustenta qualquer treinamento: dado relevante, estruturado, validado, higienizado, duplicado e suficientemente diverso.

Agora começa a segunda decisão: como treinar. Posso seguir por uma API gerenciada na nuvem ou por um caminho local com LoRA. O módulo 3 explora o primeiro caminho. O módulo 4 explora o segundo.

Essa separação precisa permanecer explícita. Aprovar fine-tuning não significa que API gerenciada seja automaticamente a melhor técnica. A escolha só fica completa quando eu consigo comparar custo, operação, tempo, controle e dependência de infraestrutura dos dois caminhos.

O risco de depender de um único provedor

Durante a preparação desta disciplina, dois provedores que eram escolhas naturais para fine-tuning via API mudaram de disponibilidade.

O material registra que o self-service de fine-tuning da OpenAI entrou em processo de descontinuação para novos acessos e para contas inativas, com desligamento completo previsto em etapas. Também registra que a API pública do Gemini para fine-tuning já havia sido encerrada anteriormente.

O ponto não é concluir que esses provedores são ruins. É exatamente o contrário: eram nomes centrais quando se falava em fine-tuning gerenciado. Se serviços tão conhecidos podem mudar em um intervalo relativamente curto, então a disponibilidade precisa fazer parte da arquitetura.

Um tutorial antigo pode continuar correto conceitualmente e ainda assim apontar para um fluxo que já não existe. Quem constrói produção precisa verificar a capacidade real do provedor na data em que vai usar, não apenas confiar em documentação histórica, post de blog ou exemplo de curso.

Diligência de provedor como parte da arquitetura

A Camila, no caso da Amplitude Seguros, não aprova um provedor apenas porque ele possui documentação e uma API funcional. A pergunta é de continuidade operacional.

Se o time precisar retreinar o modelo seis meses depois, o serviço continuará disponível? O modelo base ainda estará suportado? O acesso continuará self-service? O contrato e as permissões continuarão adequados?

Essa diligência evita que uma solução tecnicamente correta se torne operacionalmente frágil. Fine-tuning não é um evento isolado. O modelo será versionado, reavaliado e possivelmente retreinado. Se a infraestrutura usada no primeiro ciclo desaparecer, a manutenção futura fica comprometida.

A escolha de provedor precisa, portanto, considerar disponibilidade, integração, suporte, governança e estratégia de saída.

O provedor adotado na disciplina

Para os próximos capítulos, o caminho escolhido é o Vertex AI dentro do ecossistema do Google Cloud, apresentado no material sob a marca associada à plataforma empresarial do Gemini.

A escolha não acontece por falta de alternativas. O conteúdo também cita provedores como Azure OpenAI, Together AI, Fireworks e Mistral como opções que continuam relevantes em cenários de fine-tuning gerenciado.

O Vertex AI é escolhido principalmente por integração com o restante do stack utilizado na disciplina. Armazenamento, identidade, permissões, faturamento, cotas e operação de modelos ficam dentro do mesmo ecossistema de nuvem.

Essa integração reduz a quantidade de peças novas introduzidas apenas para treinar. Para a Amplitude Seguros, isso significa que o modelo ajustado pode ser tratado como mais um artefato governado dentro da infraestrutura de cloud do projeto.

O processo genérico de fine-tuning gerenciado

Mesmo com diferenças entre provedores, o processo de fine-tuning por API comercial costuma seguir cinco passos.

Primeiro, preparar o dataset no formato exato esperado pelo provedor. Segundo, enviar esse dataset para um armazenamento acessível ao serviço de treinamento. Terceiro, configurar o job escolhendo o modelo base e os hiperparâmetros. Quarto, executar o treinamento de forma gerenciada na nuvem. Quinto, monitorar o job até chegar a sucesso ou falha.

Essa sequência é suficientemente estável para ser tratada como arquitetura. O que muda de provedor para provedor é o contrato concreto: formato dos dados, SDK, nomes dos parâmetros, modelos disponíveis e superfície de monitoramento.

Passo 1: converter o dataset

O pipeline do módulo 2 já trabalha com um esquema canônico de quatro campos: instrução, entrada, saída e metadata.

Esse esquema canônico não precisa ser o mesmo formato consumido pelo provedor. Na verdade, cada serviço pode exigir uma estrutura específica, muitas vezes baseada em JSONL com papéis como system, user e model.

É justamente aqui que o investimento em um formato intermediário bem definido se paga. Eu não quero reescrever gate de relevância, OCR, PII scrubbing, deduplicação, balanceamento e validação toda vez que trocar de provedor.

Eu mantenho o pipeline de dados independente e adiciono apenas uma etapa de adaptação no final. Se o fornecedor mudar, substituo o conversor de saída, não o processo inteiro de preparação.

Passo 2: fazer upload

Depois de converter o dataset, preciso disponibilizá-lo para o serviço de treinamento.

Em um ambiente gerenciado, isso normalmente significa armazenar o arquivo em um bucket ou serviço equivalente na nuvem. O caminho precisa ser endereçável e o serviço de treinamento precisa possuir permissão de leitura.

Esse detalhe aparentemente simples já cria uma fronteira operacional importante. Um arquivo correto pode falhar no treinamento se o serviço não tiver acesso ao bucket, se a região estiver incompatível ou se a política de identidade estiver incorreta.

Por isso, upload não é apenas copiar um arquivo. Ele faz parte do contrato entre pipeline de dados, armazenamento e infraestrutura de treinamento.

Passo 3: configurar o job

Com o dataset disponível, eu configuro o job.

Essa etapa inclui a escolha do modelo base e dos hiperparâmetros, como número de épocas, taxa de aprendizado e tamanho de batch.

Cada provedor define faixas e nomes próprios, mas a função conceitual desses parâmetros continua a mesma. Eles controlam quanto e como o modelo será ajustado a partir dos exemplos.

Nos próximos capítulos, essa configuração ganha profundidade. Neste momento, o importante é entender que ela faz parte da declaração do job e não da lógica de preparação do dataset.

Passo 4: treinamento gerenciado

Depois da configuração, o provedor executa o treinamento.

Eu não administro diretamente a GPU, o processo distribuído ou o carregamento do modelo. A infraestrutura é responsabilidade do serviço gerenciado.

Isso reduz a complexidade operacional para quem quer treinar sem manter um ambiente próprio. Em troca, eu aceito depender de fila, disponibilidade, modelos suportados, limites e preços definidos pelo provedor.

Mesmo com datasets de centenas de exemplos, o treinamento não é instantâneo. O serviço precisa alocar recurso, carregar o modelo base e executar as épocas definidas. O tempo relevante fica concentrado nessa etapa.

Passo 5: monitorar o job

A última etapa é acompanhar o status.

Um job costuma passar por estados como pending, running e, por fim, succeeded ou failed.

Falha também é informação útil. O problema pode estar em formato inválido, hiperparâmetro rejeitado, cota, permissão ou outra validação feita pelo provedor.

O monitoramento pode acontecer por consultas periódicas, notificações ou pelo console do serviço. O ponto é que o pipeline de produção precisa saber identificar quando o job terminou e qual artefato foi produzido.

Escala de tempo do processo

Preparar e subir o dataset tende a ser rápido para a escala usada na Amplitude Seguros. Mesmo depois de deduplicar e balancear, estamos falando de centenas de exemplos, não de milhões.

Configurar o job também é uma operação curta. O tempo real está no treinamento.

Isso prepara uma comparação importante para o módulo 4. Tanto na nuvem quanto localmente, um dataset pequeno pode levar minutos. A diferença estrutural não é apenas a duração. É quem controla e paga por aquele tempo.

No serviço gerenciado, existe uma fila e uma infraestrutura compartilhada. No treinamento local, a máquina está sob meu controle, desde que eu tenha hardware suficiente. Essa comparação será feita com números mais adiante.

Cada treinamento gera um artefato versionado

Outro detalhe importante é que o resultado de um treinamento não deve ser tratado como um modelo que muda silenciosamente.

Cada novo ciclo com dataset ou hiperparâmetros diferentes gera um novo checkpoint ou endpoint versionado.

Isso significa que retreinar não deveria substituir o artefato anterior sem rastreabilidade. A produção precisa saber qual versão está atendendo, com qual dataset e com quais configurações.

Essa discussão se conecta ao módulo de versionamento e documentação de modelos, em que o modelo fine-tunado passa a ser tratado como artefato de software e de machine learning, não como um estado invisível do provedor.

Vertex AI dentro do ecossistema Google Cloud

O Vertex AI adotado nesta disciplina não deve ser confundido com a antiga API pública do Gemini que oferecia fine-tuning diretamente por API key.

Aqui eu trabalho com uma plataforma gerenciada de Google Cloud, integrada a IAM, armazenamento, cotas, faturamento e registro de modelos.

O fluxo passa por um dataset armazenado em bucket, um job supervisionado de treinamento e um endpoint publicado ao final.

Também existe um ponto importante sobre o ciclo de vida dos modelos. O próprio material registra que versões específicas do Gemini possuem datas de retirada anunciadas. Portanto, o nome exato do modelo usado em uma demonstração pode mudar quando o aluno executar o exercício.

O processo arquitetural continua válido: escolher uma versão vigente com suporte a fine-tuning supervisionado e verificar a documentação atual antes de montar o job.

Validação prática antes dos dados reais

Antes de levar o dataset real da Amplitude Seguros ao provedor, eu valido o caminho completo com dados sintéticos.

A demonstração utiliza 32 exemplos separados dos conjuntos empregados no módulo 2. A intenção não é avaliar a qualidade do modelo. É verificar a infraestrutura e compatibilidade do pipeline.

O job passa por pending, depois running e chega a succeeded em pouco menos de meia hora. O custo estimado fica abaixo de um centavo de dólar e um endpoint é publicado ao final.

Essa execução confirma quatro pontos que documentação nenhuma garante sozinha: a conversão do schema foi aceita, o upload para o bucket funcionou com as permissões corretas, os hiperparâmetros foram aceitos e o status reportado pela API correspondeu ao que aparecia no console do Google Cloud.

É esse tipo de teste ponta a ponta que eu quero antes de colocar dados reais no pipeline.

Governança precisa ser reavaliada ao trocar de provedor

A troca de provedor também reabre a questão de governança.

No módulo de preparação, eu já reduzi PII antes do treinamento e tratei Saúde Empresarial com atenção adicional por envolver dado sensível. Mesmo assim, uma base legal e um DPA aprovados para um serviço não se transferem automaticamente para outro.

Cada provedor possui contrato, condições de processamento e responsabilidades próprias. Se o motivo da troca foi descontinuação de mercado, a revisão ainda precisa acontecer.

Esse ponto reconecta o módulo 3 ao gate apresentado no módulo 1 e ao tratamento de dados do módulo 2. Governança não é uma etapa que eu marco uma vez e esqueço. Ela acompanha mudanças de infraestrutura e de fluxo de dados.

Do dataset preparado ao treinamento gerenciado

Com o dataset já preparado e o provedor validado, o próximo passo é juntar as duas partes do pipeline.

O mesmo conjunto que passou por relevância, extração, validação, higienização, deduplicação e balanceamento será convertido para o formato esperado pelo Vertex AI, enviado para o armazenamento e utilizado na criação de um job real.

Os próximos capítulos entram em upload, acompanhamento, hiperparâmetros e monitoramento com chamadas reais contra o provedor.

A partir daqui, fine-tuning deixa de ser apenas uma decisão e também deixa de ser apenas um problema de preparação de dados. Ele passa a ser um pipeline operacional completo, no qual qualidade do dataset, contrato com o provedor, versionamento e monitoramento precisam funcionar juntos.

NA PRÁTICA

- Represente o processo gerenciado em cinco passos: converter, fazer upload, configurar o job, treinar e monitorar.
- Registre quais verificações podem ser feitas localmente antes de consumir recursos do provedor.
- Inclua a diligência de provedor e a reavaliação de governança como parte da arquitetura do treinamento.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-1)”?
- Qual relação existe entre Fine-tuning gerenciado e Vertex AI no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No módulo anterior, eu fechei a preparação do dataset da Amplitude Seguros.
- Seria natural começar este capítulo enviando imediatamente esse dataset para algum provedor.
- Essa mudança de perspectiva é importante porque dataset pronto não significa infraestrutura de treinamento garantida.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — Fine-Tuning via API (upload e train) (PT-2)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-2)”.
- Relacionar Esquema canônico e Gate de confiança de OCR às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Esquema canônico	• Gate de confiança de OCR
• Upload versionado	• Pending / Running / Succeeded
• Tokens e custo	• Orquestração de nuvem

Conteúdo da aula

No capítulo anterior, eu apresentei o caminho de fine-tuning gerenciado que passa a ser utilizado nesta disciplina. Depois de discutir a mudança de disponibilidade de alguns provedores self-service e de validar o Vertex AI ponta a ponta com um pequeno conjunto sintético, chegou a hora de trabalhar com os dados que representam os casos da Amplitude Seguros em uma escala de treinamento mais próxima da prática.

Até aqui, nenhuma execução de fine-tuning havia utilizado o dataset real dos casos de Auto e Saúde Empresarial. Agora eu avanço sobre os dois primeiros passos do processo genérico que defini no módulo 3.1: preparar o dataset no formato exato esperado pelo provedor e fazer o upload para a nuvem, acompanhando o job até o final.

A demonstração não fica restrita à criação do job. Eu também valido a conversão do esquema canônico, aplico o gate de confiança de OCR, escalo o pipeline formal do módulo 2, reavalio Saúde Empresarial com o mesmo Decision Framework utilizado anteriormente e consulto o estado de um job real diretamente na API do Google Cloud.

Por que o job já existe antes da gravação

O job apresentado nesta aula foi criado antes da gravação. Isso não significa que o resultado é simulado. A razão é operacional: um fine-tuning real leva dezenas de minutos e não faria sentido manter uma aula inteira parada esperando a infraestrutura concluir o treinamento.

O que eu executo ao vivo é uma consulta de status real contra a API do provedor. Essa consulta utiliza um token de acesso gerado pelo gcloud na máquina local para autenticar a chamada e provar que a conta possui autorização para acessar o projeto.

A distinção é importante. O treinamento foi real, com dataset real do exercício, e a API consultada durante a aula também é real. Apenas a espera entre a criação do job e seu término foi retirada da gravação.

Do esquema canônico ao formato do Vertex AI

O módulo 2 definiu deliberadamente um esquema canônico genérico com instrução, entrada, saída e metadata. O objetivo era separar a preparação de dados da formatação específica de um fornecedor.

Agora esse desenho começa a pagar o investimento. Para o formato consumido pelo Vertex AI, instrução e entrada são combinadas no turno do usuário. A saída esperada vira o turno do modelo, preservada como o JSON exato que o extrator precisa aprender a devolver.

Cada exemplo passa, portanto, a ter dois turnos: um de usuário e outro de modelo, cada um contendo suas partes de conteúdo.

Essa camada de adaptação é pequena porque limpeza, gate de relevância, deduplicação, balanceamento e demais decisões de preparação continuam independentes do provedor. Se o mercado mudar novamente, eu altero o conversor final e não o pipeline inteiro.

Onde começa a orquestração real de nuvem

Existe uma fronteira clara entre o que ainda roda localmente e o que começa a consumir a infraestrutura do Google Cloud.

A conversão do dataset e o gate de OCR acontecem localmente. Só depois, quando obtenho credenciais e começo a interagir com os serviços de nuvem, entro de fato na orquestração do Vertex AI.

Para quem quiser configurar o próprio projeto, o material disponibiliza um guia opcional de setup do GCP. Esse tipo de separação é útil porque permite testar conversão e regras de qualidade sem depender de credenciais, cota ou custo de nuvem.

Testando a conversão antes do upload

O primeiro bloco da execução roda treze testes automatizados.

Esses testes confirmam que cada exemplo convertido sempre possui dois turnos, que o turno do usuário carrega instrução e entrada completas e que o turno do modelo contém exatamente o JSON esperado.

Também verifico os casos inválidos. Exemplos sem instrução, entrada ou saída são rejeitados antes de qualquer upload.

Isso impede que um erro simples de preparação chegue até o job gerenciado e só seja descoberto depois de alocar recurso de treinamento.

Além disso, a suíte testa o gate de confiança de OCR para garantir que os exemplos sejam aprovados, encaminhados para revisão humana ou ignorados pelo motivo correto.

Convertendo exemplos reais de Auto e Saúde Empresarial

Depois dos testes, eu converto dois exemplos reais do dataset, um de Amplitude Auto e outro de Saúde Empresarial.

No turno do usuário, entra literalmente a instrução combinada com o texto bruto do documento. Eu não faço uma edição manual específica para deixar o exemplo mais fácil para o modelo.

No turno do modelo, a saída é o contrato estruturado esperado para cada caso.

Em Auto, os campos são seguro, placa e valor. Em Saúde Empresarial, os campos são beneficiário, procedimento e valor.

Esse teste visual da conversão ajuda a verificar se o esquema canônico está realmente chegando ao formato do provedor sem perder informação ou alterar o contrato que o modelo precisa aprender.

O gate de confiança de OCR

O campo de confiança de OCR que havia sido preparado anteriormente passa agora a ter uma função concreta no pipeline.

Nos documentos reais escaneados pelo Tesseract no módulo 2.1, a confiança fica aproximadamente entre 94% e 96%. Todos esses exemplos passam pelo gate.

Quando eu simulo um documento mais degradado, com confiança de 62%, o pipeline não o envia automaticamente para treinamento. Ele sinaliza o exemplo para revisão humana.

A regra evita transformar erro de OCR em comportamento aprendido pelo modelo. Se a leitura do documento já é duvidosa, eu não deveria tratá-la como gabarito confiável.

Os 200 exemplos utilizados no job de treinamento desta aula não passam por esse gate porque foram gerados como texto sintético. Eles não carregam uma confiança de OCR real. Nesse caso, a ausência da métrica é interpretada como não aplicável, e não como baixa qualidade.

O gate continua disponível para futuras ingestões de documentos reais.

Escalando o pipeline do módulo 2

Os 34 exemplos didáticos utilizados anteriormente eram suficientes para provar que o pipeline de limpeza funcionava, mas não representavam o volume desejado para um job de treinamento real.

A resposta não foi inventar outro método. Eu escalo exatamente o mesmo pipeline formal.

Continuo utilizando MinHash com LSH para deduplicação, amostragem por temperatura para balancear fontes e entropia de Shannon para medir diversidade.

A execução começa com 305 exemplos brutos, distribuídos entre seis oficinas no caso de Auto e cinco clínicas no caso de Saúde Empresarial. Duas fontes novas são acrescentadas em cada cenário para ampliar a cobertura.

Depois da deduplicação, restam 300 exemplos. O balanceamento por temperatura reduz o conjunto para 200 exemplos finais, sendo 120 de Auto e 80 de Saúde Empresarial.

A diversidade efetiva de fontes também melhora. Em Auto, o número efetivo sobe de aproximadamente 5,516 para 5,723. Em Saúde Empresarial, passa de aproximadamente 4,140 para 4,706.

É a mesma evidência formal utilizada no módulo 2.2, agora aplicada a um conjunto aproximadamente seis vezes maior.

Reavaliando Saúde Empresarial nove meses depois

Existe uma aparente contradição que eu preciso resolver antes de treinar.

No módulo 1.2, Saúde Empresarial foi reprovado por falta de dados suficientes. No módulo 1.3, o Decision Framework recomendou esperar nove meses. Se eu simplesmente incluísse o caso no treinamento agora, estaria ignorando a própria decisão ensinada anteriormente.

Por isso, eu rodo novamente o mesmo framework.

A reavaliação não cria uma lógica nova. O script importa diretamente o Decision Framework já existente e mantém os mesmos pesos, limiar e três perguntas que já estavam verdes. A única dimensão atualizada é a pergunta 3, sobre suficiência dos dados.

O score anterior era 0,35. A projeção usada anteriormente considerava crescimento de 0,03 por mês. Nove meses depois, o cálculo chega a 0,62, acima do limiar de 0,6.

O volume mensal também cresce de 1.200 para aproximadamente 1.862 casos, seguindo a mesma taxa usada na análise anterior.

Dessa vez, nenhuma pergunta permanece vermelha. O caso entra no treinamento porque o mesmo critério que o reprovou antes agora o aprova com dados atualizados. A régua não foi reduzida para caber na demonstração.

Um bug real no gerador sintético

Ao aumentar o volume do dataset sintético, aparece um problema que vale ser tratado como parte da aula.

O gerador original utilizava listas de nomes, placas e valores com o mesmo tamanho, quarenta itens. Em uma fonte com mais de quarenta exemplos, os índices começavam a ciclar juntos.

O exemplo de posição 41 podia repetir nome, placa e valor do exemplo 1, mudando apenas a data. Isso criava colisões artificiais que o detector de duplicidade interpretava como duplicatas.

A correção foi aumentar o período antes de repetição. Os nomes passaram a ser formados por combinações de primeiro nome e sobrenome vindos de pools com tamanhos diferentes, produzindo mais de 1.500 combinações antes de repetir.

Depois da correção, os falsos positivos desaparecem e apenas os cinco pares de duplicatas plantados propositalmente são detectados.

A lição é generalizável. Um gerador sintético com listas curtas pode contaminar a métrica que deveria provar a qualidade do pipeline. Por isso, eu sempre valido as duplicatas do conjunto bruto antes de confiar no resultado.

Upload versionado para o bucket

Com o dataset convertido e salvo em JSONL, o upload para o armazenamento do Google Cloud é simples.

O arquivo é enviado para um bucket específico do projeto e depois referenciado na criação do job de tuning junto com o modelo base e os hiperparâmetros.

Na execução utilizada para esta disciplina, o job foi configurado com Gemini 2.5 Flash, três épocas e um nome de exibição associado ao conjunto de 200 exemplos de Auto e Saúde Empresarial.

O bucket já existia de testes anteriores, mas o dataset atual foi enviado como um novo objeto.

Essa escolha é deliberada. Cada versão de dataset precisa manter seu próprio caminho. Eu não sobrescrevo silenciosamente o arquivo anterior, porque um job que falha ou produz um resultado inesperado precisa continuar apontando para exatamente os dados usados naquele treinamento.

Criando e identificando o job

Depois do upload, o job é criado no serviço de tuning do Vertex AI.

A execução real recebe um identificador próprio e entra inicialmente no estado Pending.

A partir desse momento, o trabalho pesado roda na infraestrutura do provedor. Minha máquina não precisa permanecer responsável pelo treinamento.

O que eu preciso manter é a capacidade de consultar o job, registrar seu identificador e acompanhar as transições de estado até chegar ao resultado final.

Interpretando Pending, Running e Succeeded

Os estados do job deixam de ser abstrações e passam a ter significado operacional.

Pending representa a fila. O provedor já recebeu a solicitação, mas ainda não alocou os recursos necessários para iniciar o treinamento.

Running significa que o treinamento está efetivamente em execução. As épocas estão sendo processadas e os pesos estão sendo atualizados conforme os lotes de exemplos.

Succeeded indica que todas as etapas terminaram sem erro e que o artefato resultante foi publicado.

Existe também o caminho de falha. Se o job chegar a Failed, a resposta da API pode trazer informações como formato inválido do dataset, cota excedida ou hiperparâmetro fora da faixa aceita.

Em produção, esses estados determinam se o sistema apenas espera ou se alguém precisa agir.

Consultando o job real pela API

A parte central da demonstração é uma consulta real ao endpoint REST do serviço de AI Platform do Google Cloud.

A chamada autenticada devolve o estado do job, o modelo base, o nome de exibição, os timestamps de início e fim e os endpoints gerados.

O job apresentado chega a Succeeded.

A duração total foi de aproximadamente 45 minutos e 42 segundos. O tempo é superior ao piloto anterior, que utilizou 32 exemplos e levou cerca de 28 minutos e 32 segundos, porque agora o conjunto de treinamento é aproximadamente seis vezes maior.

Essa comparação também mostra que o custo de um job gerenciado não deve ser estimado apenas pela quantidade de minutos visíveis. O Vertex AI utilizado na demonstração contabiliza o treinamento pelos tokens processados.

Tokens, custo e artefatos resultantes

O job processa aproximadamente 27.353 tokens de treinamento.

O custo registrado para essa execução fica na faixa de cinco a onze centavos de dólar.

Ao final, existem dois artefatos importantes: o modelo ajustado e o endpoint publicado para inferência.

Esses artefatos fecham o ciclo de upload e treinamento. O próximo problema passa a ser acompanhar métricas e configurar hiperparâmetros com mais profundidade.

A consulta de status realizada manualmente nesta aula será automatizada posteriormente, e o endpoint publicado será tratado como artefato versionado e documentado nos capítulos seguintes.

Da validação do job aos hiperparâmetros

O ponto deste capítulo não é apenas mostrar que um job chegou a Succeeded.

Eu valido que o esquema canônico pode ser convertido sem reescrever o pipeline, que exemplos incompletos são bloqueados, que OCR de baixa confiança pode exigir revisão humana e que o mesmo mecanismo de deduplicação e balanceamento escala para um conjunto maior.

Também mostro que uma decisão anterior pode ser reavaliada sem alterar a lógica do framework. Saúde Empresarial só passa a ser treinado porque o score de dados realmente cruza o limiar depois do período de espera projetado.

Por fim, a execução conecta dataset, armazenamento, job gerenciado, estados operacionais e artefatos publicados numa única trilha reproduzível.

No próximo capítulo, eu entro nos hiperparâmetros. O foco passa a ser entender o que cada configuração controla, como escolher épocas e taxa de aprendizado para o volume disponível e o que as estatísticas de monitoramento do provedor conseguem, e não conseguem, revelar sobre o treinamento.

NA PRÁTICA

- Converta exemplos do esquema canônico para o formato esperado pelo provedor escolhido no conteúdo.
- Defina um gate de confiança de OCR antes do upload e registre o motivo de uma eventual rejeição.
- Acompanhe o ciclo de estados do job e registre identificador, tokens, custo e artefatos resultantes quando disponíveis.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-2)”?
- Qual relação existe entre Esquema canônico e Gate de confiança de OCR no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu apresentei o caminho de fine-tuning gerenciado que passa a ser utilizado nesta disciplina.
- Até aqui, nenhuma execução de fine-tuning havia utilizado o dataset real dos casos de Auto e Saúde Empresarial.
- A demonstração não fica restrita à criação do job.

[↑ Voltar ao mapa da disciplina](#)

Aula 3 — Fine-Tuning via API (upload e train) (PT-3)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-3)”.
- Relacionar Épocas e Underfitting e overfitting às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Épocas	• Underfitting e overfitting
• Learning rate multiplier	• Adapter size
• Validação pré-rede	• Pedido versus aplicado

Conteúdo da aula

No capítulo anterior, eu executei o pipeline de upload e acompanhamento do fine-tuning gerenciado sobre 200 exemplos da Amplitude Auto e da Amplitude Saúde Empresarial. O job chegou ao estado Succeeded depois de aproximadamente 45 minutos e 42 segundos. Isso prova que o processo funciona operacionalmente, mas ainda falta responder uma pergunta mais importante: ele foi configurado corretamente para o volume e o tipo de dado que eu tinha?

Agora eu quero entender o que cada hiperparâmetro controla, por que um valor pode fazer sentido para um dataset pequeno e deixar de fazer sentido quando a escala muda, e quais mecanismos preciso colocar ao redor da API para não depender cegamente da validação do provedor.

O foco passa de conseguir criar um job para conseguir operá-lo com critérios. Isso inclui escolher hiperparâmetros, validar a configuração antes de gastar recurso, confirmar o que o provedor realmente aplicou e interpretar as estatísticas devolvidas pela plataforma.

Os três hiperparâmetros centrais

A configuração deste módulo trabalha com três hiperparâmetros principais.

O primeiro é o epoch count, que define quantas vezes o modelo percorre o dataset completo. Três épocas significam que cada exemplo participa do processo de ajuste três vezes.

O segundo é o learning rate multiplier. Ele controla a intensidade com que cada exemplo influencia o ajuste. No serviço gerenciado, eu não defino uma taxa absoluta do zero, mas multiplico uma taxa base interna calibrada pelo provedor.

O terceiro é o adapter size, ligado à capacidade da camada adaptativa treinada. Como o fine-tuning eficiente não precisa alterar todos os parâmetros do modelo base, essa camada aprende a correção necessária para o comportamento específico que quero especializar.

Épocas e o equilíbrio entre underfitting e overfitting

O número de épocas precisa equilibrar dois riscos.

Com poucas passagens pelo dataset, o modelo pode não aprender suficientemente o padrão. Esse é um cenário de underfitting.

Com épocas demais, principalmente em datasets pequenos, o modelo pode memorizar exemplos específicos em vez de aprender uma estrutura geral. Esse é o overfitting.

As três épocas utilizadas com os 200 exemplos procuram um ponto intermediário. Não existe uma regra universal dizendo que três é sempre o valor correto. É uma escolha coerente com o tamanho atual do dataset e com o comportamento relativamente estreito que estou treinando.

Relação entre tamanho do dataset e número de épocas

Uma heurística útil é observar a relação entre volume e repetição.

Quando o dataset é menor, mais épocas podem ajudar até certo limite porque cada padrão aparece poucas vezes. Quando o dataset cresce, os padrões se repetem entre exemplos diferentes e muitas épocas podem se tornar desnecessárias.

Por isso, 200 exemplos com três épocas formam um ponto intermediário. Com um conjunto muito maior, uma ou duas épocas poderiam bastar. Com poucos exemplos, eu poderia testar mais passagens, sempre acompanhado por avaliação adequada.

Essa relação é um princípio de partida, não uma fórmula. Em comparações experimentais, às vezes faz sentido manter o número de épocas fixo para isolar outra variável.

Learning rate multiplier

O learning rate multiplier pode ser confundido com uma taxa de aprendizado tradicional.

No Vertex AI, ele atua sobre uma taxa base interna. Um valor próximo de 1 preserva um ritmo próximo do calibrado pelo provedor. Um valor maior intensifica o ajuste a cada exemplo.

Na configuração usada na disciplina, o multiplicador é 5. Com poucas centenas de exemplos e três épocas, um multiplicador baixo poderia produzir alteração pouco perceptível, deixando o modelo fine-tunado muito parecido com o modelo base.

Por outro lado, um valor alto demais pode desestabilizar o treinamento. A decisão precisa considerar volume de dados, objetivo do fine-tuning e quantidade de passagens pelo conjunto.

Adapter size e capacidade da camada treinável

O adapter size representa outra dimensão da decisão.

Um rank baixo restringe a camada adaptativa a menos graus de liberdade e consegue aprender um padrão estreito com menor custo de memória e treinamento.

No caso da Amplitude Seguros, o comportamento é específico: receber um documento e produzir uma saída estruturada curta com três campos principais. Por isso, a configuração utilizada trabalha com adapter size 4.

Ranks maiores, como 8 ou 16, podem capturar comportamentos mais complexos, mas custam mais parâmetros treináveis, memória e tempo. No módulo de treinamento local, essa relação volta com mais profundidade porque o rank passa a afetar diretamente a memória disponível na máquina.

O problema real encontrado no Vertex AI

A parte mais importante da aula surge de um teste deliberadamente inválido.

Antes de gravar, eu quis verificar se o Vertex AI rejeitava rapidamente um hiperparâmetro impossível. Para isso, tentei criar um job com epoch count igual a zero.

A expectativa seria receber a falha de validação antes de qualquer recurso ser alocado. Não foi o que aconteceu.

O provedor aceitou o job, levou a execução para Running e o campo inválido acabou ausente na configuração aplicada. Em vez de recusar explicitamente o valor, a plataforma seguiu sem deixar o problema evidente na criação.

Assim que identifiquei a divergência, cancelei o treinamento. O job ficou ativo por cerca de quarenta segundos. O custo foi pequeno, mas real.

Silêncio da API não significa configuração correta

Se uma API aceita uma chamada sem erro, isso não prova que todos os parâmetros foram aplicados da forma esperada.

Em uma automação semanal de retreino, um bug simples poderia zerar o epoch count e continuar criando jobs durante semanas. O primeiro sintoma talvez surgisse apenas no comportamento ruim do modelo publicado.

Por isso, ausência de reclamação do provedor não pode ser tratada como confirmação. Configuração precisa ser validada por quem chama a API e auditada depois que o job existe.

Primeira camada de proteção: validar antes da rede

A primeira proteção é uma validação local de hiperparâmetros, executada antes de qualquer chamada de rede e antes de qualquer possibilidade de custo.

Na implementação demonstrada, o epoch count precisa estar entre 1 e 20 e ser inteiro. Valores zero, negativos ou fracionários são rejeitados. O learning rate multiplier também recebe uma faixa aceitável, de 0,1 a 10.

Essa validação transforma uma falha que poderia consumir infraestrutura em um erro local imediato e barato. O epoch count zero que escapou da API real passa a ser bloqueado antes da requisição.

Segunda camada de proteção: comparar pedido e aplicado

A segunda proteção acontece depois da criação do job.

Eu comparo os hiperparâmetros solicitados com aquilo que o próprio objeto do job informa ter aplicado.

Essa verificação existe porque nenhuma validação local prevê perfeitamente todas as regras internas do provedor. Se um campo desaparecer, for alterado ou receber outro valor, a divergência precisa aparecer.

As duas camadas não são redundantes. Uma evita erros antes de gastar. A outra protege contra comportamento inesperado depois que o provedor recebeu o pedido.

Comparação numérica e uma pegadinha de API

Algumas APIs do Google devolvem determinados números como texto. Se eu comparar diretamente o número 3 com o texto "3" usando igualdade estrita, o resultado será falso, embora os valores sejam equivalentes.

Por isso, a função de comparação tenta converter os dois lados para número antes de decidir se existe divergência. Quando isso não faz sentido, volta para comparação direta.

Sem esse cuidado, o monitor poderia acusar erros em jobs corretos e levar a cancelamentos desnecessários.

Checklist antes da criação do job

Antes de criar um treinamento real, eu consigo resumir a análise em três perguntas.

O número de épocas é compatível com o tamanho do dataset ou foi apenas copiado de algum tutorial? O learning rate multiplier faz sentido para o volume e a intensidade de ajuste desejada? O adapter size é o menor que representa o comportamento necessário?

Essas perguntas levam pouco tempo, mas evitam que uma configuração ruim entre em uma fila de treinamento apenas porque a API aceitou o payload.

Onde começa a chamada real ao Google Cloud

Assim como na aula anterior, existe uma fronteira entre lógica local e orquestração real de nuvem.

Testes, validações e comparação de configuração podem rodar localmente sem custo. A partir do momento em que o script obtém o token de acesso e consulta o Google Cloud, as chamadas dependem da autenticação real do projeto.

O token é obtido pelo gcloud autenticado na máquina. Se a ferramenta não estiver conectada ao projeto correto, essa parte da demonstração não executa.

Os testes automatizados desta etapa

A ferramenta de hiperparâmetros e monitoramento começa com nove testes automatizados.

Eles confirmam que a configuração real do módulo 3.2 passa pela validação, que epoch count igual a zero é rejeitado, que valores negativos ou não inteiros falham e que a comparação entre solicitado e aplicado detecta divergência de valor e campo ausente.

Outro teste cobre a equivalência entre texto e número, evitando que uma representação diferente da API seja interpretada como alteração real.

Em seguida, o script reproduz o bloqueio do epoch count zero localmente, mostrando o erro antes de qualquer acesso à rede.

Consultando os hiperparâmetros realmente aplicados

Depois dos testes locais, eu consulto novamente o job real do módulo 3.2.

A resposta confirma os valores aplicados. O learning rate multiplier está em 5, o adapter size em 4 e o epoch count corresponde ao valor esperado.

Pedido e aplicado batem.

Esse resultado é tão importante quanto observar o caso inválido. A finalidade da comparação é produzir evidência de que a configuração executada corresponde ao que foi aprovado antes da criação do job. Os hiperparâmetros passam a fazer parte da trilha auditável do treinamento.

Monitoramento sem curva de loss

Quem está acostumado a treinamento local pode esperar que a API devolva uma curva de loss durante o job.

Nesse fluxo do Vertex AI, essa curva não está disponível pela API usada na demonstração. O provedor expõe estatísticas do dataset calculadas durante a criação do job.

A API também aceita um dataset de validação separado, capaz de produzir estatísticas de validação durante o treinamento. Eu deixo esse recurso de fora de propósito porque a avaliação rigorosa do modelo fine-tunado fica para o módulo 5.

Aqui eu quero aprender a operar e monitorar o job sem misturar ainda a discussão completa de baseline e qualidade final.

Estatísticas reais do dataset de treinamento

A consulta do job devolve aproximadamente 27.353 tokens cobráveis para os 200 exemplos.

Também consigo observar estatísticas de tamanho de entrada e saída. As entradas ficam na ordem de uma centena de tokens por exemplo, enquanto as saídas ficam perto de 27 tokens em média, coerentes com um JSON pequeno de três campos.

Essas estatísticas não substituem a curva de loss. Elas ajudam a entender o volume efetivamente processado, a relação entre entrada e saída e a base sobre a qual o provedor calcula a cobrança.

Também funcionam como verificação indireta. Se uma tarefa que deveria produzir respostas curtas aparecesse com saídas muito grandes, eu teria motivo para voltar ao dataset.

A configuração é contextual, não definitiva

Três épocas, learning rate multiplier 5 e adapter size 4 fazem sentido para o dataset e para a tarefa que estou treinando agora. Eles não se tornam defaults universais.

Com um dataset significativamente maior, eu provavelmente reduziria o número de épocas. Com uma tarefa mais ampla, talvez o rank do adaptador precisasse crescer.

Em um experimento comparativo, eu também poderia manter determinados valores fixos para observar o efeito de apenas uma variável.

O princípio é sempre o mesmo: hiperparâmetro precisa ser uma decisão registrada e relacionada ao experimento, não um número copiado sem contexto.

Da operação manual para automação

A próxima evolução é automatizar o fluxo que agora já possui as proteções necessárias.

O script poderá preparar e fazer upload do dataset, validar hiperparâmetros antes de enviar, criar o job, registrar os valores pedidos, consultar o que foi aplicado e monitorar sozinho as mudanças de estado até sucesso ou falha.

Essa automação só é segura porque as validações foram construídas antes. Automatizar um pipeline que confia cegamente na API apenas faria o erro silencioso acontecer com mais eficiência.

No próximo capítulo, o foco passa a ser transformar essas chamadas isoladas em um fluxo JavaScript que cria e acompanha o job sem exigir consultas manuais.

NA PRÁTICA

- Defina valores pretendidos para épocas, learning rate multiplier e adapter size, justificando a escolha.
- Valide a configuração antes da chamada de rede e compare posteriormente os valores pedidos com os efetivamente aplicados.
- Registre qualquer parâmetro silenciosamente ignorado pela API como falha de configuração a ser tratada.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-3)”?
- Qual relação existe entre Épocas e Underfitting e overfitting no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu executei o pipeline de upload e acompanhamento do fine-tuning gerenciado sobre 200 exemplos da Amplitude Auto e da Amplitude Saúde Empresarial.
- Agora eu quero entender o que cada hiperparâmetro controla, por que um valor pode fazer sentido para um dataset pequeno e deixar de fazer sentido quando a escala muda, e quais mecanismos preciso colocar ao redor da API para não depender cegamente da validação do provedor.
- O foco passa de conseguir criar um job para conseguir operá-lo com critérios.

[↑ Voltar ao mapa da disciplina](#)

Aula 4 — Fine-Tuning via API (upload e train) (PT-4)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-4)”.
- Relacionar Automação e Confirmação às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Automação	• Confirmação
• Idempotência	• Polling
• Backoff exponencial	• Callback
• Injeção de dependência	

Conteúdo da aula

No capítulo anterior, eu deixei prontas duas proteções importantes para o fine-tuning gerenciado. Antes de criar qualquer job, os hiperparâmetros passam por validação local. Depois que o job existe, eu comparo aquilo que foi solicitado com o que o provedor realmente aplicou. Também passei a consultar estatísticas reais do treinamento em vez de assumir que a ausência de erro significava que tudo estava correto.

Até aqui, porém, cada etapa foi executada de forma isolada, sempre com um comando específico. Agora eu junto essas peças em uma única automação em JavaScript. O objetivo é transformar conversão, validação, upload, criação de job e acompanhamento em um fluxo sequencial que consiga operar sozinho sem perder os gates de segurança que construímos até aqui.

Automatizar não significa remover todo controle humano. Pelo contrário. Quanto mais etapas um script consegue executar sem supervisão, mais importante fica decidir quais ações são seguras para rodar sozinhas e quais precisam de uma confirmação explícita antes de provocar custo ou criar um recurso real.

Os cinco passos da automação

A função de orquestração encadeia cinco passos.

Primeiro, converto os exemplos para o formato esperado pelo Gemini. Segundo, valido os hiperparâmetros. Terceiro, envio o dataset para o bucket. Quarto, crio o job de fine-tuning. Quinto, acompanho o job até que ele chegue a um estado terminal.

Os dois primeiros passos reutilizam a mesma lógica apresentada nos capítulos anteriores, agora incorporada ao script de automação. A intenção não é mudar o comportamento, mas provar que as peças continuam funcionando quando deixam de ser chamadas manualmente e passam a fazer parte de uma cadeia única.

A ordem também importa. Eu não posso criar um job sem que o dataset tenha sido enviado. Não posso fazer upload antes de converter o formato. E não faz sentido acompanhar um job que ainda não existe.

Sequência simples e falha rápida

A orquestração é propositalmente sequencial. Cada etapa espera a anterior terminar antes de continuar.

Essa estrutura já fornece uma garantia importante. Se o upload falhar por permissão incorreta, indisponibilidade de rede ou destino inválido, a execução é interrompida ali. O código não avança para a criação do job com um estado incompleto.

O mesmo vale para a validação. Se um exemplo estiver incompleto, o processo para antes dos hiperparâmetros. Se um hiperparâmetro estiver inválido, o processo para antes do upload.

Esse comportamento de fail fast evita que uma falha pequena no início do pipeline se transforme em um recurso criado de forma incorreta no final. Muitas vezes, uma função assíncrona simples e sequencial fornece uma garantia mais clara do que uma orquestração excessivamente paralela para um fluxo em que cada etapa depende da anterior.

A etapa que não deve rodar sem confirmação

Entre os cinco passos, a criação do job recebe tratamento diferente.

Converter dados, validar parâmetros, consultar estado e até repetir um upload para o mesmo destino são operações que podem ser automatizadas com risco relativamente controlado. Criar um job de fine-tuning, porém, gera um recurso cobrável.

Por isso, a função de criação só executa a chamada de rede quando recebe uma confirmação explícita igual a true nas opções.

Sem essa confirmação, o erro é lançado localmente antes de qualquer chamada ao provedor. A mensagem lembra inclusive o incidente do módulo anterior, quando um epoch count inválido foi aceito pela API e chegou a iniciar um treinamento real.

A regra é deliberada: automação de verdade não é ausência de supervisão. É colocar supervisão exatamente onde o impacto justifica.

O padrão de confirmação antes de uma ação cara

A trava utilizada aqui segue um princípio que aparece em outras ferramentas de infraestrutura.

Um exemplo conhecido é a separação entre planejar e aplicar uma mudança em ferramentas de infraestrutura como código. Primeiro eu vejo o que será feito. Depois confirmo a execução que realmente altera recursos.

Em diferentes serviços de cloud também existem modos de simulação ou dry run pelo mesmo motivo. A confirmação explícita no script de fine-tuning é uma versão mínima desse padrão aplicada à criação de um job que custa dinheiro.

Essa ideia também se conecta aos gates dos módulos anteriores. No módulo 2.1, um documento precisava provar relevância antes de entrar no dataset. No módulo 2.2, o balanceamento nunca podia alocar mais exemplos do que uma fonte possuía. Quando uma ação pode causar dano real, eu valido antes, não depois.

Idempotência parcial do pipeline

A automação também precisa considerar o que acontece quando o script é executado duas vezes.

No passo de upload, repetir a operação para o mesmo caminho substitui o objeto no destino e mantém o mesmo resultado final. Nesse sentido, o upload é idempotente para o caso apresentado.

A criação do job não possui essa mesma propriedade. Cada chamada bem-sucedida cria um novo treinamento e pode gerar nova cobrança.

É por isso que a confirmação explícita fica justamente nesse ponto. Não faria sentido exigir intervenção humana para toda etapa segura, mas seria perigoso permitir que uma nova execução acidental criasse outro job silenciosamente.

O problema do polling constante

Depois de criar o job, a automação precisa acompanhar o status até o final.

Consultar a API a cada segundo seria simples, mas ineficiente. Na maior parte do tempo, um job continua no mesmo estado e não existe informação nova para recuperar.

Eu preciso de uma estratégia que consulte com frequência suficiente para continuar responsiva, mas sem martelar o endpoint de status desnecessariamente.

A solução utilizada é o backoff exponencial.

Backoff exponencial com teto

A primeira consulta acontece imediatamente depois da criação do job. Se o estado ainda não for terminal, o script passa a esperar entre as próximas consultas.

O primeiro intervalo é de cinco segundos. Depois, cada espera é multiplicada por 1,5 até chegar a um teto de sessenta segundos.

A sequência cresce aproximadamente de 5 para 7,5, depois 11,25, 16,875 e continua aumentando até estabilizar em sessenta segundos.

O crescimento evita chamadas excessivas. O teto evita o problema contrário. Sem um limite, um job que demorasse horas poderia fazer o intervalo crescer tanto que a automação passaria muitos minutos sem perceber que o treinamento já havia terminado.

Com o teto de sessenta segundos, mesmo em um job longo, o atraso máximo para detectar o estado final fica limitado a aproximadamente um minuto.

Por que usar fator 1,5

O fator 1,5 não é o único possível.

Se eu dobrasse o intervalo em cada rodada, chegaria ao teto mais rápido e faria menos chamadas. Em compensação, ficaria mais tempo sem consultar justamente na faixa intermediária, onde um job de duração média pode terminar.

O crescimento por 1,5 é mais gradual. Ele consome algumas consultas adicionais no início, mas mantém maior responsividade até alcançar o platô de sessenta segundos.

Para os jobs desta disciplina, que levam dezenas de minutos, a maior parte do tempo de acompanhamento acaba acontecendo já nesse teto. O backoff reduz o ruído no início e depois mantém uma frequência previsível até o término.

Callback para tornar o acompanhamento observável

O loop de monitoramento utiliza também um callback.

Callback é uma função fornecida ao processo para ser chamada automaticamente quando um evento acontece. Neste caso, ela recebe atualizações conforme o script consulta o estado do job.

Isso permite registrar Pending, Running e Succeeded sem que alguém precise consultar manualmente a API.

Sem callback, o acompanhamento poderia funcionar internamente e ainda parecer travado para quem observa o terminal. A automação estaria viva, mas muda por fora.

Com atualizações explícitas, eu consigo enxergar que o processo continua consultando, qual foi o último estado recebido e quando houve uma transição relevante. Observabilidade também faz parte de uma automação confiável.

A fronteira entre execução local e Google Cloud

O script mantém uma separação clara entre a parte local e a parte que realmente toca a nuvem.

Conversão e validação rodam sem custo e sem dependência de credenciais externas.

A partir do upload, criação e acompanhamento, o código passa a interagir com o Google Cloud de verdade. É nessa região que entram autenticação, permissões, bucket, API e recursos cobráveis.

Manter essa fronteira explícita facilita testes e reduz o risco de uma alteração em lógica local disparar acidentalmente alguma ação remota.

Os dezoito testes automatizados

A demonstração executa dezoito testes automatizados divididos em grupos.

O grupo de upload verifica se o comando de cópia para o bucket é montado corretamente, se um arquivo que não é JSONL é rejeitado e se o destino precisa começar com o esquema esperado do Google Cloud Storage.

Outro grupo testa a trava de confirmação. A criação é bloqueada quando não existe confirmação, continua bloqueada com opções vazias e só é liberada quando a confirmação é explicitamente verdadeira.

Também são repetidos testes de conversão para o formato Gemini e validação de hiperparâmetros, garantindo que incorporar a mesma lógica em uma automação maior não alterou o comportamento validado anteriormente.

Testando a orquestração de ponta a ponta

Um grupo novo testa a função que coordena os cinco passos.

O teste confirma que conversão, validação, upload, criação e acompanhamento acontecem na ordem correta e que o resultado de cada etapa é devolvido pela orquestração.

Também verifica os caminhos de falha. Um exemplo incompleto interrompe o fluxo antes da validação de hiperparâmetros. Um hiperparâmetro inválido interrompe antes do upload. A confirmação continua sendo exigida na etapa de criação.

Esses testes são importantes porque uma coleção de funções corretas não garante automaticamente que a composição delas também esteja correta. O comportamento da cadeia precisa ser testado como uma unidade.

Testando o backoff

O cálculo do backoff também recebe testes próprios.

Eles confirmam que o fator de crescimento é aplicado corretamente, que o teto de sessenta segundos nunca é ultrapassado e que, depois de várias rodadas, o intervalo realmente satura no limite esperado.

Isso evita que uma mudança aparentemente pequena no cálculo transforme o monitor em um cliente agressivo demais ou, no extremo contrário, em um processo que demora muito para perceber o fim do job.

Testando lógica assíncrona sem rede nem espera real

O grupo mais importante testa o loop assíncrono de acompanhamento sem utilizar a rede e sem esperar segundos de verdade.

Para isso, a função de consulta e a função de espera são injetáveis. Durante o teste, eu substituo ambas por versões falsas e controláveis.

Um teste simula um job que já está em estado terminal na primeira consulta. Nesse caso, nenhuma espera deve acontecer.

Outro simula a sequência Pending, Running, Running e Succeeded. O loop consulta quatro vezes e espera exatamente três vezes, uma entre cada estado não terminal.

Um terceiro verifica que o callback recebe todos os estados intermediários na ordem correta, e não apenas o resultado final.

Injeção de dependência

A técnica que permite esses testes recebe aqui um nome formal: injeção de dependência.

Em vez de a função de acompanhamento estar presa a uma implementação fixa de consulta de rede e a uma espera real, essas operações entram como dependências substituíveis.

Em produção, eu forneço a função que consulta a API e a função que realmente espera. No teste, forneço versões falsas que retornam exatamente os estados desejados e concluem em milissegundos.

Sem essa separação, testar o loop exigiria conexão real com o provedor ou minutos de espera entre consultas. A suíte ficaria lenta, cara e sujeita a falhas externas.

Com a injeção de dependência, a lógica assíncrona continua sendo a mesma, mas a infraestrutura é trocada por uma versão controlável. O teste se torna rápido e determinístico.

Acompanhando o job real

Depois da suíte, a demonstração volta ao job real criado no módulo 3.2.

Como o treinamento já terminou, a primeira consulta encontra diretamente o estado Succeeded. O loop identifica que o estado é terminal e devolve o endpoint publicado sem executar qualquer espera adicional.

Se o job ainda estivesse em andamento, o mesmo código continuaria sozinho, consultando com backoff e chamando o callback até chegar ao estado final.

A diferença é que agora não preciso digitar novos comandos para cada consulta. O acompanhamento deixou de ser uma sequência manual e virou comportamento do próprio pipeline.

Automação segura antes de versionar o modelo

Ao final desta etapa, eu tenho um pipeline que converte o dataset, valida configuração, prepara o upload, cria o job somente com confirmação explícita e acompanha o treinamento até obter o endpoint final.

Ele também possui testes para caminhos felizes e de falha, proteção contra criação acidental de recursos e uma estratégia de polling que respeita a API sem perder responsividade.

O mesmo fluxo está disponível em JavaScript e também em Python com a lógica de dependências substituíveis para teste.

O endpoint produzido pelo job do módulo 3.2 agora é o artefato que segue para o próximo passo. No capítulo seguinte, ele será versionado e documentado, fechando o ciclo do fine-tuning via API e preparando a missão prática em que esse pipeline poderá ser adaptado para outro dataset e outro contexto de negócio.

NA PRÁTICA

- Modele os cinco passos da automação com falha rápida e confirmação explícita antes da ação cara.
- Implemente ou descreva polling com backoff exponencial e um callback para tornar o acompanhamento observável.
- Separe dependências externas para que a lógica assíncrona possa ser testada sem rede e sem espera real.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-4)”?
- Qual relação existe entre Automação e Confirmação no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu deixei prontas duas proteções importantes para o fine-tuning gerenciado.
- Até aqui, porém, cada etapa foi executada de forma isolada, sempre com um comando específico.
- Automatizar não significa remover todo controle humano.

[↑ Voltar ao mapa da disciplina](#)

Aula 5 — Fine-Tuning via API (upload e train) (PT-5)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-5)”.
- Relacionar Linhagem e SHA-256 às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

CONCEITOS-CHAVE

• Linhagem	• SHA-256
• Model Card	• Model Registry
• Versionamento	• Rastreabilidade
• Preference Tuning	

Conteúdo da aula

No capítulo anterior, eu fechei a automação de ponta a ponta do fine-tuning via API. O pipeline já consegue converter o dataset, validar hiperparâmetros, fazer upload, proteger a criação do job com uma confirmação explícita e acompanhar o treinamento até o endpoint ser publicado. Só que um endpoint isolado ainda diz muito pouco sobre o modelo que está em produção.

Daqui a alguns meses, alguém pode perguntar qual dataset treinou aquele endpoint, quais hiperparâmetros foram realmente aplicados, qual modelo base serviu de ponto de partida e em que momento aquele artefato passou a existir. Se eu não tiver essas respostas registradas, o modelo está funcionando, mas a operação perdeu a linhagem.

Por isso, o último passo deste módulo é versionar e documentar. Eu quero transformar um endpoint publicado em um artefato rastreável, ligado ao conteúdo exato do dataset, à configuração aplicada, ao job que gerou o modelo e à linha do tempo em que tudo aconteceu.

Por que um endpoint não é documentação

Um endpoint é apenas um identificador operacional. Ele permite chamar o modelo, mas não conta a história que levou até aquele resultado.

Em produção, isso se torna um risco. O mesmo projeto pode retreinar várias vezes com versões diferentes do dataset. Pode trocar hiperparâmetros, mudar o modelo base ou publicar um novo endpoint. Se essas relações não estiverem registradas, fica difícil explicar por que o comportamento mudou ou reproduzir um treinamento anterior.

Essa dor é uma das razões pelas quais MLOps se tornou uma disciplina própria dentro da engenharia de machine learning. Versionar código com Git é um problema conhecido há décadas. Versionar código, dataset, experimento, modelo e publicação como uma única linhagem ainda exige disciplina específica.

O que precisa ser versionado

Para qualquer modelo fine-tunado que chegue à produção, eu quero amarrar pelo menos quatro elementos.

O primeiro é o dataset de treinamento, identificado pelo conteúdo e não apenas pelo nome do arquivo.

O segundo são os hiperparâmetros realmente aplicados. Essa distinção importa porque o módulo 3.3 já mostrou que aquilo que eu peço para a API e aquilo que o provedor registra como aplicado podem divergir.

O terceiro é o modelo base, ou seja, o checkpoint do provedor utilizado como ponto de partida.

O quarto é o resultado do treinamento: modelo ajustado, endpoint ou endpoints publicados e identificadores reais do job.

Esses elementos já existiam espalhados pelos capítulos anteriores. Nesta etapa, eu reúno todos em um artefato único e verificável.

O modelo base também precisa de linhagem

Registrar o modelo base é mais importante do que parece.

Provedores aposentam versões. Um modelo que hoje aceita novos jobs pode deixar de aceitar treinamento no futuro.

O material utiliza o Gemini 2.5 Flash como exemplo do modelo base empregado ao longo do módulo e registra uma data de retirement anunciada para essa versão.

Mesmo que o modelo ajustado continue publicado, o time precisa saber qual base o originou. Isso afeta reprodutibilidade, retreino e comparação com versões futuras.

Sem esse campo, um modelo fine-tunado pode continuar funcionando enquanto a sua origem já não está mais disponível para um novo ciclo equivalente.

Nome de arquivo não é versão

O ponto mais importante desta aula está no dataset.

Um nome como dataset-final-v2.jsonl não prova o conteúdo daquele arquivo. O arquivo pode ser renomeado, movido ou sobrescrito mantendo exatamente o mesmo nome.

Se alguém corrigir um exemplo e salvar por cima do arquivo original, o caminho pode continuar idêntico enquanto o conteúdo que existe hoje já não é o mesmo que treinou o modelo publicado.

Nesse cenário, documentar apenas o nome do arquivo cria uma falsa sensação de versionamento. Eu sei onde o arquivo está, mas não sei se ele ainda representa o conteúdo usado no treinamento.

Hash SHA-256 como identificador de conteúdo

A solução utilizada é calcular um hash SHA-256 do conteúdo do dataset.

Para um mesmo arquivo, byte por byte, o resultado é sempre o mesmo valor hexadecimal de 64 caracteres. Se qualquer parte do conteúdo mudar, mesmo uma alteração pequena, o hash muda completamente.

Isso transforma o conteúdo em identificador de versão.

O princípio é semelhante ao versionamento por conteúdo utilizado em ferramentas como Git e em imagens de container. O que define a identidade não é o nome amigável, mas uma assinatura derivada do conteúdo real.

Para o dataset de 200 exemplos utilizado nos módulos anteriores, o hash fica registrado no artefato de versionamento. Enquanto aquele arquivo permanecer exatamente igual, o identificador permanece o mesmo.

Detectando alterações silenciosas no dataset

O hash resolve um cenário comum em produção.

Imagine que alguém descubra um erro de digitação em um exemplo, corrija o arquivo e faça upload novamente para o mesmo caminho do bucket.

Sem hash, um Model Card antigo pode continuar apontando para o mesmo nome de dataset e dar a impressão de que aquele conteúdo foi utilizado no treinamento original.

Com hash, a comparação é objetiva. O Model Card guarda o identificador calculado no momento do treinamento. Se o arquivo atual tiver outro hash, eu sei imediatamente que o conteúdo mudou.

Não preciso depender da memória de quem fez a alteração ou de uma convenção de nomes aplicada corretamente.

Model Card como documentação estruturada

A documentação final é organizada em um Model Card.

O termo não nasce nesta disciplina. O material referencia o trabalho Model Cards for Model Reporting, publicado por pesquisadores do Google, que formalizou a ideia de documentar modelos de maneira estruturada para apoiar entendimento e uso responsável.

Aqui eu uso uma versão mais enxuta e focada no pipeline que construímos.

O Model Card inclui identificação do job, modelo ajustado, endpoint, estado final, modelo base, dataset e hash, hiperparâmetros aplicados, estatísticas do dataset e linha do tempo de criação e conclusão.

A função desse documento é deixar de falar de um modelo genérico e passar a falar daquele modelo específico, treinado naquele conteúdo, daquela forma e naquele momento.

Do arquivo Markdown ao Model Registry

Para a escala desta disciplina, um arquivo Markdown é suficiente para registrar a linhagem.

Em um ambiente maior, o mesmo princípio costuma ser formalizado em um registro de modelos, como Vertex AI Model Registry, MLflow ou outra solução equivalente.

O registro automatiza parte do trabalho. Cada novo treino pode gerar uma versão, associar métricas, armazenar artefatos e apoiar promoção ou rollback entre versões publicadas.

A diferença é principalmente de automação e escala. Conceitualmente, os campos que eu estou registrando agora são os mesmos que um model registry precisa conhecer para manter a proveniência do modelo.

Rastreabilidade do dado e rastreabilidade do modelo

Esse comportamento conecta o módulo 3 ao módulo 2.

Durante a preparação de dados, eu preservei o texto OCR bruto e os metadados para conseguir rastrear a origem de um exemplo estranho.

Agora eu aplico a mesma disciplina ao modelo inteiro.

Rastrear de onde veio o exemplo e rastrear de onde veio o modelo são dois níveis da mesma prática. Se o resultado em produção estiver errado, eu preciso conseguir descer da versão publicada para o job, do job para o dataset e do dataset para os exemplos que o compõem.

O que permanece local e o que consulta a nuvem

Quase toda a lógica desta etapa pode rodar localmente.

Calcular hash, duração, custo estimado, montar a ficha de versionamento e gerar o Model Card não exige uma nova chamada ao provedor.

A única parte que toca de fato o Google Cloud é a consulta ao job já existente para recuperar os identificadores e os valores registrados pela plataforma.

Essa separação mantém o processo barato e testável. O artefato de documentação pode ser regenerado ou validado sem criar nenhum novo recurso de treinamento.

Os treze testes automatizados

A ferramenta de versionamento executa treze testes divididos em três grupos.

O primeiro grupo valida o hash de conteúdo. O mesmo arquivo calculado duas vezes precisa produzir exatamente o mesmo resultado. O valor precisa ter 64 caracteres hexadecimais e conteúdos diferentes precisam produzir hashes diferentes.

Para provar a última condição, o teste cria um arquivo temporário com conteúdo diferente e compara as assinaturas.

O segundo grupo testa a ficha de versionamento. Um job completo deve gerar uma ficha válida. Um job sem identificador obrigatório precisa ser rejeitado. A validação de completude também precisa falhar quando um campo essencial, como o endpoint, é removido propositalmente.

O terceiro grupo cobre custo e Model Card, verificando duração, presença do hash, identificadores do modelo ajustado, endpoint e dados financeiros registrados.

A ficha gerada a partir do job real

Depois dos testes, a ferramenta calcula o hash real do dataset de 200 exemplos utilizado no treinamento dos módulos 3.2 a 3.4.

Em seguida, consulta o job real e monta a ficha completa.

Ela registra o modelo base, o dataset e seu hash, epoch count igual a 3, learning rate multiplier igual a 5 e adapter size igual a 4. São os mesmos valores que o módulo 3.3 confirmou como efetivamente aplicados.

Também entram as estatísticas do treinamento, incluindo os 200 exemplos e aproximadamente 27.353 tokens cobráveis, além dos identificadores do modelo ajustado e do endpoint publicado.

A ficha não reconstrói o passado por suposição. Cada campo vem de uma decisão ou de um resultado observado nos capítulos anteriores.

Por que registrar criação e conclusão

O versionamento guarda os horários de criação e conclusão, não apenas uma duração total.

Essa diferença ajuda na investigação de incidentes.

Se o endpoint começar a se comportar de forma inesperada em determinada data, eu posso cruzar o momento em que o modelo mudou com deploys, alterações de configuração ou incidentes reportados no mesmo intervalo.

Sem timestamps absolutos, eu sei que um job durou quarenta e poucos minutos, mas não consigo posicionar essa mudança na linha do tempo operacional do sistema.

Custo estimado e custo real

A ficha também separa custo estimado de custo real.

O custo estimado utiliza uma faixa de referência associada a GPU para permitir comparação com o caminho local que será explorado no módulo 4.

Ele não representa a fatura do Vertex AI, porque o serviço gerenciado utilizado aqui cobra o treinamento por tokens e não simplesmente por hora de GPU.

Por isso, a ficha também registra o custo efetivamente verificado no billing do Google Cloud para o job, no exemplo apresentado, R\$2,39.

Manter os dois valores lado a lado ajuda a comparar alternativas sem confundir estimativa de infraestrutura própria com cobrança real do serviço gerenciado.

O Model Card como artefato do pipeline

Ao final da execução, o Model Card é gerado como um arquivo real.

Ele pode acompanhar o modelo no repositório, na documentação da equipe ou ser usado como base para um registro de modelos mais formal.

O mesmo dataset produz o mesmo hash independentemente de eu calcular a assinatura em JavaScript ou Python. O versionamento por conteúdo depende do arquivo, não da linguagem usada para processá-lo.

Isso permite transportar a prática para qualquer ambiente de treinamento sem alterar o princípio.

A missão prática que fecha o módulo 3

A missão prática deste módulo reúne tudo o que foi construído até aqui.

Primeiro, eu converto o dataset da missão anterior, ou outro dataset do meu contexto, para o formato esperado pelo provedor escolhido.

Segundo, valido os hiperparâmetros antes da chamada e comparo o que foi solicitado com o que realmente foi aplicado depois da criação do job.

Terceiro, implemento acompanhamento com backoff exponencial em JavaScript e incluo pelo menos um teste automatizado que não dependa de rede, utilizando injeção de dependência.

Quarto, gero uma ficha de versionamento com hash do dataset e os campos mínimos de um Model Card.

A entrega não exige obrigatoriamente gastar dinheiro com um job real. Se o orçamento não permitir, é válido simular a criação e documentar exatamente o que teria sido enviado, incluindo hiperparâmetros, hash e custo estimado.

Também é possível analisar por escrito jobs já publicados na disciplina, comparando configurações e resultados sem iniciar novo treinamento.

Rastreabilidade antes mesmo de gastar

Documentar uma decisão de não executar também faz parte da rastreabilidade.

Se eu registro qual dataset seria utilizado, qual é o hash, quais hiperparâmetros seriam enviados e qual custo estimo, outra pessoa consegue entender posteriormente qual decisão foi considerada e por que ela não avançou.

Rastreabilidade não serve apenas para explicar o que já aconteceu. Ela também serve para registrar de forma verificável o que estava planejado antes de uma ação cara ou irreversível.

O que continua valendo no treinamento local

No próximo módulo, a infraestrutura muda. O treino sai da API gerenciada e entra na máquina local com técnicas de Parameter Efficient Fine-Tuning.

O rigor de engenharia, porém, não muda.

O esquema canônico do módulo 2 continua útil. As validações antes de agir continuam úteis. Os testes automatizados continuam úteis. E o versionamento por conteúdo continua funcionando da mesma forma.

Quando o treinamento local gerar um adaptador, como um arquivo SafeTensor, esse artefato também pode receber um hash SHA-256. O princípio não depende de nuvem. Ele depende apenas do conteúdo que eu quero identificar de maneira verificável.

Um segundo caminho: Preference Tuning

Antes de fechar o módulo, a aula apresenta um segundo caminho disponível no Vertex AI além do Supervised Fine-Tuning utilizado até aqui: Preference Tuning.

No treinamento supervisionado, cada entrada possui uma resposta correta que funciona como gabarito. É exatamente o cenário da extração estruturada da Amplitude, em que eu quero um conjunto de campos específicos.

No Preference Tuning, a estrutura muda. Para cada prompt, eu forneço duas respostas: uma preferida e uma rejeitada. O modelo aprende a aproximar o comportamento da resposta preferida e a se afastar da alternativa rejeitada.

O material relaciona esse caminho a DPO, Direct Preference Optimization. Diferentemente do RLHF clássico, que envolve um modelo de recompensa separado e depois uma etapa de reinforcement learning, DPO otimiza diretamente a preferência entre pares de respostas.

O teste real de Preference Tuning

A técnica também é testada de verdade na disciplina.

Quarenta exemplos do módulo 3.2 são convertidos em pares de preferência. De um lado fica a extração correta. Do outro, uma resposta gerada a partir de um prompt mais fraco, sem a exigência rígida de JSON.

O job de Preference Tuning é criado no Vertex AI usando o Gemini 2.5 Flash e termina com sucesso em aproximadamente 17 minutos e 32 segundos, publicando modelo ajustado e endpoint.

O tempo menor não deve ser interpretado isoladamente como prova de superioridade. O dataset desse experimento é muito menor, com 40 exemplos, enquanto o treinamento supervisionado principal utilizou 200.

Onde Preference Tuning faz mais sentido

A própria natureza do caso ajuda a decidir qual técnica é mais adequada.

Extração de campos estruturados possui uma resposta objetiva. Para esse tipo de tarefa, Supervised Fine-Tuning é o habitat natural.

Preference Tuning ganha relevância quando qualidade é subjetiva e existe mais de uma resposta plausível, mas uma delas é preferível.

A aula cita situações como tom, voz de marca, linguagem de compliance e comunicação com cliente.

Em uma seguradora, isso poderia aparecer em comunicado de sinistro, resposta de atendimento ou linguagem utilizada em uma negativa de cobertura. Nesses casos, o jeito de responder pode importar tanto quanto o conteúdo factual.

O módulo termina, portanto, com duas ideias conectadas. Primeiro, qualquer modelo treinado precisa carregar linhagem suficiente para ser reproduzido e auditado. Segundo, escolher fine-tuning não encerra a decisão técnica. O tipo de ajuste continua dependente da natureza da tarefa e do tipo de supervisão disponível.

NA PRÁTICA

- Calcule um hash SHA-256 do conteúdo do dataset e use-o como identificador de versão.
- Monte um Model Card com linhagem do modelo base, dataset, configuração, datas, custo e resultado do treinamento.
- Explique como o artefato poderá ser registrado em um Model Registry sem depender apenas de nomes de arquivo.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-5)”?
- Qual relação existe entre Linhagem e SHA-256 no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

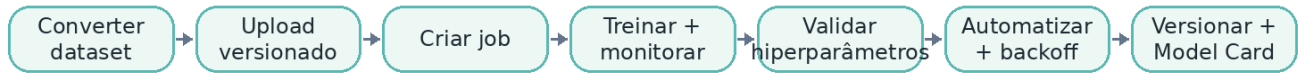
- No capítulo anterior, eu fechei a automação de ponta a ponta do fine-tuning via API.
- Daqui a alguns meses, alguém pode perguntar qual dataset treinou aquele endpoint, quais hiperparâmetros foram realmente aplicados, qual modelo base serviu de ponto de partida e em que momento aquele artefato passou a existir.
- Por isso, o último passo deste módulo é versionar e documentar.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 3

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Fine-Tuning via API \(upload e train\) \(PT-1\)](#)
- [Revisar a Aula 2: Fine-Tuning via API \(upload e train\) \(PT-2\)](#)
- [Revisar a Aula 3: Fine-Tuning via API \(upload e train\) \(PT-3\)](#)
- [Revisar a Aula 4: Fine-Tuning via API \(upload e train\) \(PT-4\)](#)
- [Revisar a Aula 5: Fine-Tuning via API \(upload e train\) \(PT-5\)](#)

[↑ Voltar ao mapa da disciplina](#)

Revisão final da disciplina

A disciplina organiza uma sequência completa: decidir se fine-tuning faz sentido, preparar o dado com gates de qualidade e governança, executar treinamento gerenciado com controles operacionais e terminar com rastreabilidade e versionamento do artefato.

FLUXO GERAL



SÍNTESE PARA REVISÃO

- Fine-tuning é tratado como uma alternativa posterior a técnicas mais baratas, e não como ponto de partida.
- O RAG é associado à recuperação de conhecimento; fine-tuning é associado a comportamento, formato e especialização repetida.
- A qualidade do treinamento depende de relevância, representatividade, governança, validação, deduplicação e diversidade do dataset.
- No treinamento gerenciado, configuração, estados do job, hiperparâmetros e automação precisam ser observáveis e testáveis.
- Dataset, modelo base, configuração, custo e resultado precisam permanecer vinculados por mecanismos de versionamento e documentação.

PERGUNTAS DE REVISÃO FINAL

- Em que situação o Decision Framework deve interromper uma recomendação de fine-tuning?
- Por que um dataset individualmente válido ainda pode ser inadequado como conjunto?
- Como a comparação entre configuração solicitada e configuração aplicada reduz risco operacional de uma API de treinamento?
- Por que o hash do dataset e o Model Card são importantes para a linhagem do modelo?
- Quais etapas do pipeline podem permanecer determinísticas mesmo quando o produto final envolve um modelo fine-tunado?

[↑ Voltar ao mapa da disciplina](#)